Advanced Problem Solving

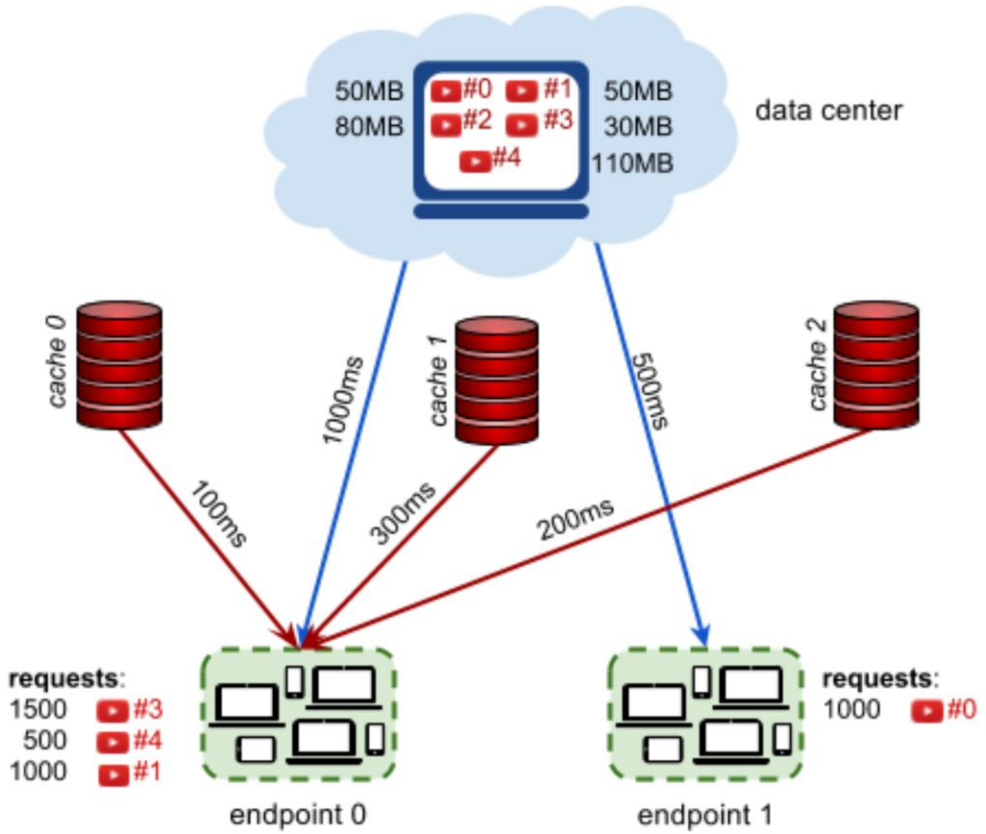
Hashcode 2017 – Présentation Finale

Justin DITER, Arno LESAGE, Hugo VIANA

(Groupe A)

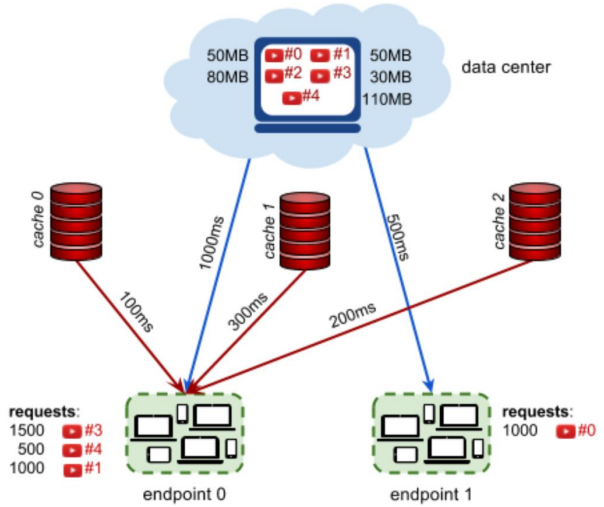
Algorithme

Greedy(Instance) :



Greedy(Instance) :

```
Pour v dans V :
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```



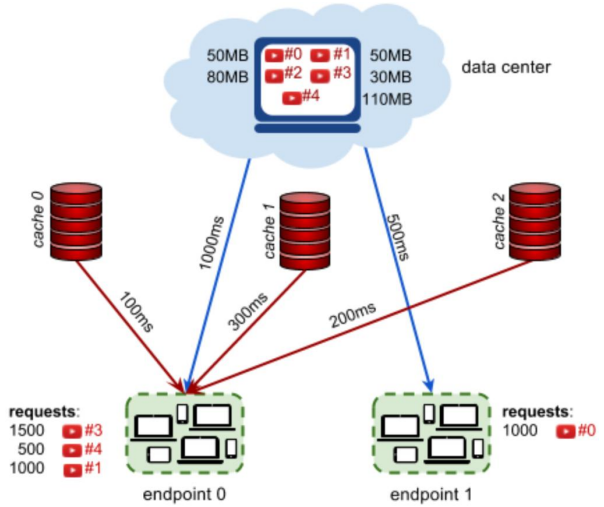
	Cache 0	Cache 1	Cache 2
Vidéo 0			
Vidéo 1			
Vidéo 2			
Vidéo 3			
Vidéo 4			

Greedy(Instance) :

Pour v dans V :

    Pour c dans C :

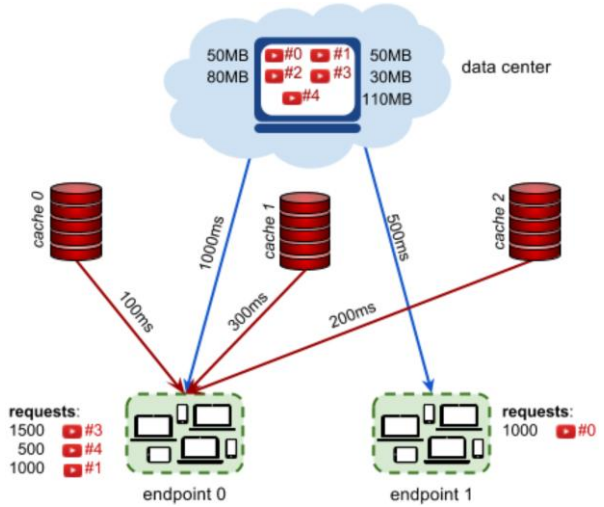
        gain[v][c] = calcul_gain(v, c)



	Cache 0	Cache 1	Cache 2
Vidéo 0	0		
Vidéo 1			
Vidéo 2			
Vidéo 3			
Vidéo 4			

Greedy(Instance) :

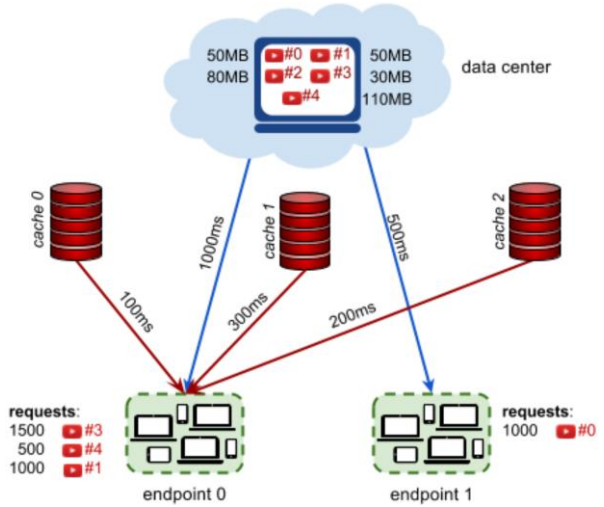
```
Pour v dans V :
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```



	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	
Vidéo 1			
Vidéo 2			
Vidéo 3			
Vidéo 4			

Greedy(Instance) :

```
Pour v dans V :
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```



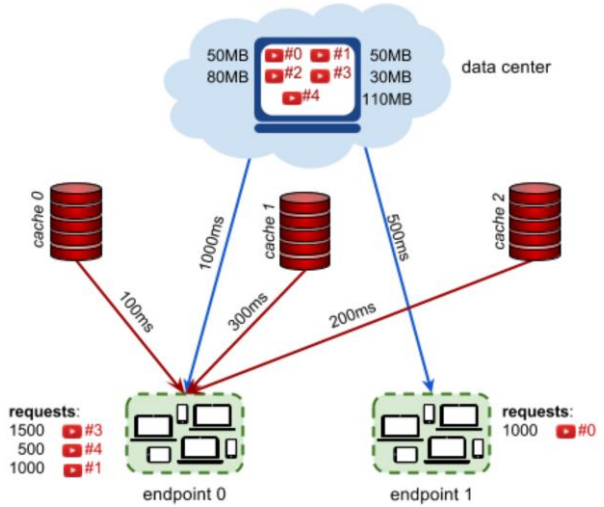
	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1			
Vidéo 2			
Vidéo 3			
Vidéo 4			

Greedy(Instance) :

Pour v dans V :

    Pour c dans C :

        gain[v][c] = calcul_gain(v, c)



$$1000(1000 - 100)$$

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000		
Vidéo 2			
Vidéo 3			
Vidéo 4			

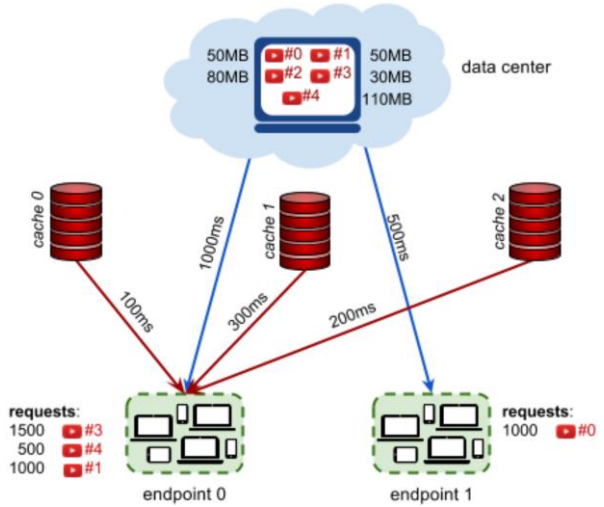
Greedy(Instance) :

Pour v dans V :

  Pour c dans C :

    gain[v][c] = calcul_gain(v, c)

$$1000(1000 - 300)$$



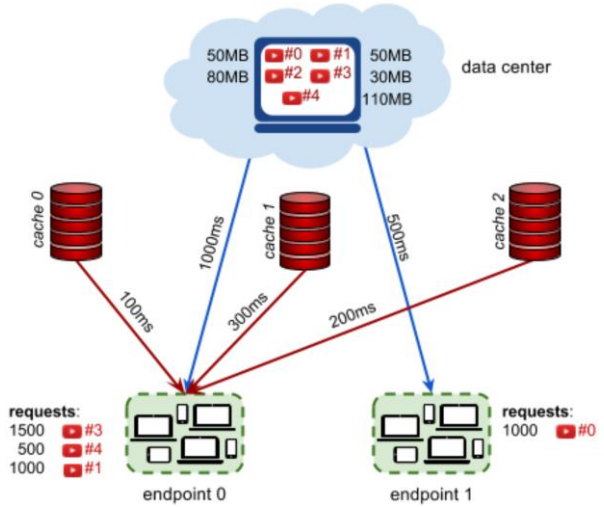
	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	
Vidéo 2			
Vidéo 3			
Vidéo 4			

Greedy(Instance) :

Pour v dans V :

  Pour c dans C :

    gain[v][c] = calcul_gain(v, c)

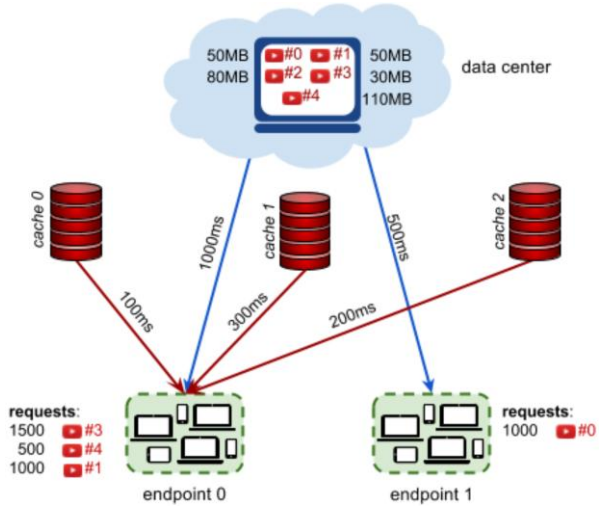


$$1000(1000 - 200)$$

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2			
Vidéo 3			
Vidéo 4			

Greedy(Instance) :

```
Pour v dans V :
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```



	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Greedy(Instance) :

Pour v dans V :

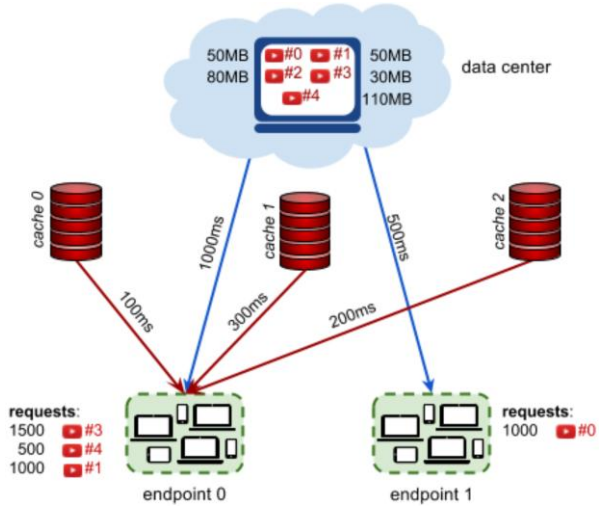
  Pour c dans C :

    gain[v][c] = calcul_gain(v, c)

$O(V)$

$O(C)$

$O(R \cdot C)$



	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Greedy(Instance) :

Pour v dans V : $O(V)$

  Pour c dans C : $O(C)$

    gain[v][c] = calcul_gain(v, c) $O(R \cdot C)$

Tant qu'il y a de la place :

  Pour v dans V :

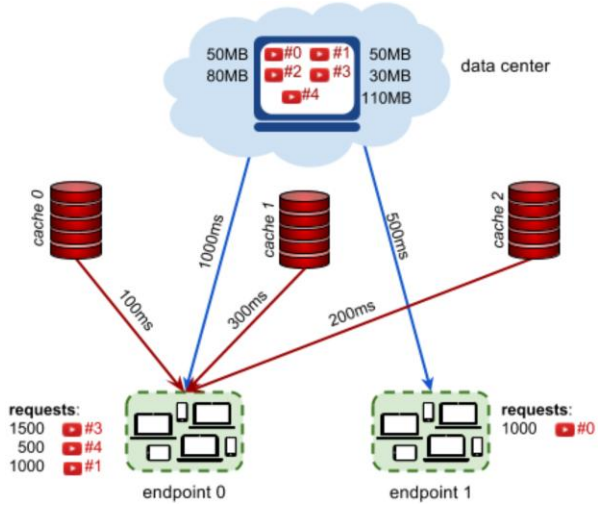
    Pour c dans C :

      Si gain[v][c] / taille(v) > best_densité :

        best_move = (v, c)

best_densité = -1

best_move = (-1, -1)



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

Greedy(Instance) :

Pour v dans V :

- Pour c dans C :
  - gain[v][c] = calcul_gain(v, c)

Tant qu'il y a de la place :

- Pour v dans V :
  - Pour c dans C :
    - Si gain[v][c] / taille(v) > best_densité :
      - best_move = (v, c)

$O(V)$

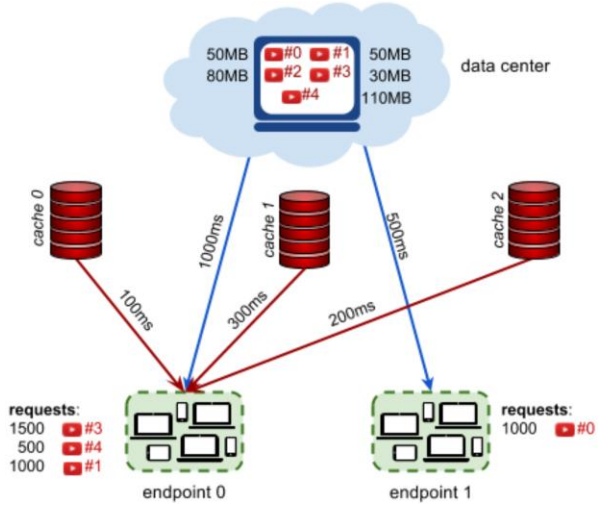
$O(C)$

$O(R \cdot C)$

best_densité = 0

best_move = (0, 0)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

Greedy(Instance) :

Pour v dans V :

- Pour c dans C :
  - gain[v][c] = calcul_gain(v, c)

Tant qu'il y a de la place :

- Pour v dans V :
  - Pour c dans C :
    - Si gain[v][c] / taille(v) > best_densité :
      - best_move = (v, c)

$O(V)$

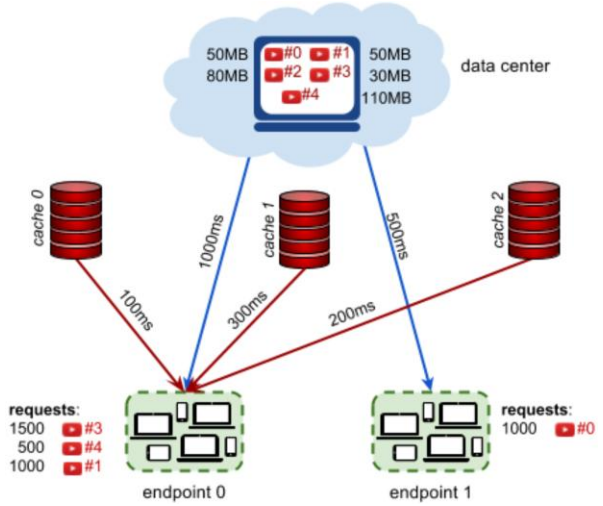
$O(C)$

$O(R \cdot C)$

best_densité = 0

best_move = (0, 0)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

Greedy(Instance) :

Pour v dans V :

- Pour c dans C :
  - gain[v][c] = calcul_gain(v, c)

Tant qu'il y a de la place :

- Pour v dans V :
  - Pour c dans C :
    - Si gain[v][c] / taille(v) > best_densité :
      - best_move = (v, c)

$O(V)$

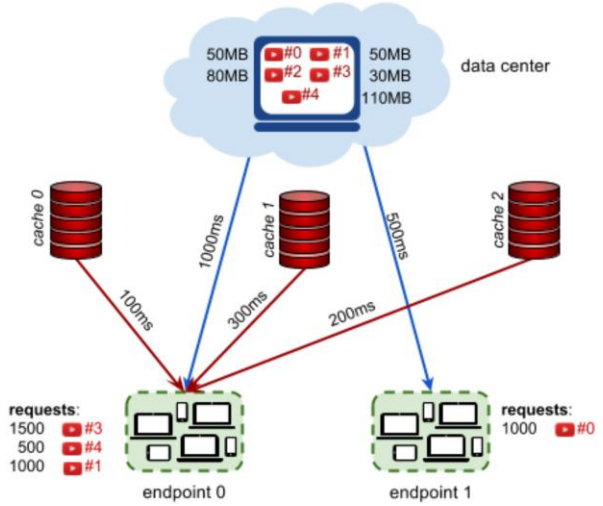
$O(C)$

$O(R \cdot C)$

best_densité = 0

best_move = (0, 0)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

Greedy(Instance) :

Pour v dans V :

- Pour c dans C :
  - gain[v][c] = calcul_gain(v, c)

Tant qu'il y a de la place :

- Pour v dans V :
  - Pour c dans C :
    - Si gain[v][c] / taille(v) > best_densité :
      - best_move = (v, c)

$O(V)$

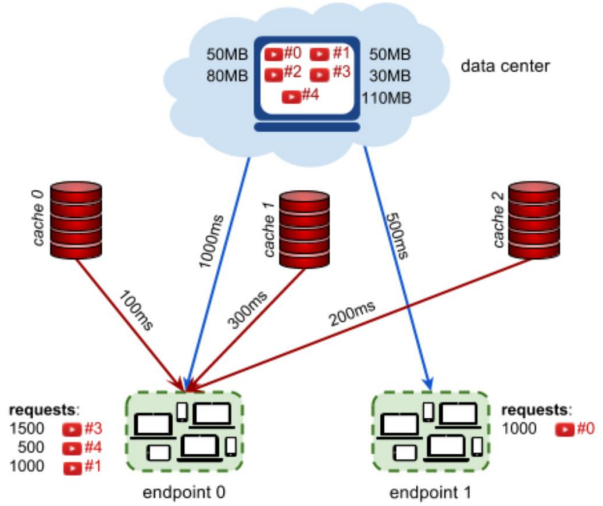
$O(C)$

$O(R \cdot C)$

best_densité = 18 000

best_move = (1, 0)

900 000/50 = 18 000



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

Greedy(Instance) :

Pour v dans V :

- Pour c dans C :
  - gain[v][c] = calcul_gain(v, c)

Tant qu'il y a de la place :

- Pour v dans V :
  - Pour c dans C :
    - Si gain[v][c] / taille(v) > best_densité :
      - best_move = (v, c)

$O(V)$

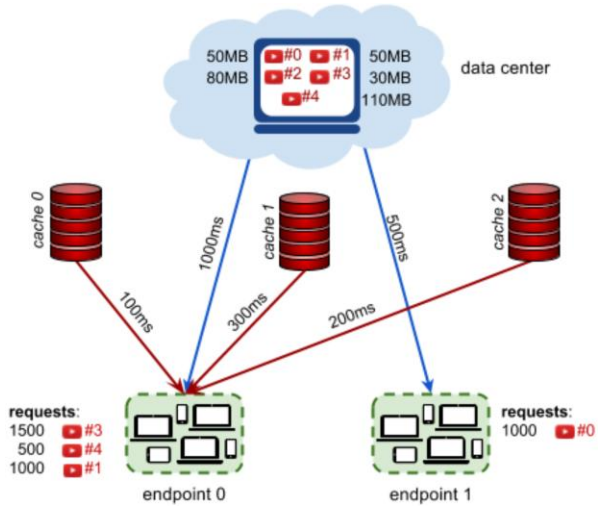
$O(C)$

$O(R \cdot C)$

best_densité = 18 000

best_move = (1, 0)

700 000/50 = 14 000



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

Greedy(Instance) :

Pour v dans V :

- Pour c dans C :
  - gain[v][c] = calcul_gain(v, c)

Tant qu'il y a de la place :

- Pour v dans V :
  - Pour c dans C :
    - Si gain[v][c] / taille(v) > best_densité :
      - best_move = (v, c)

$O(V)$

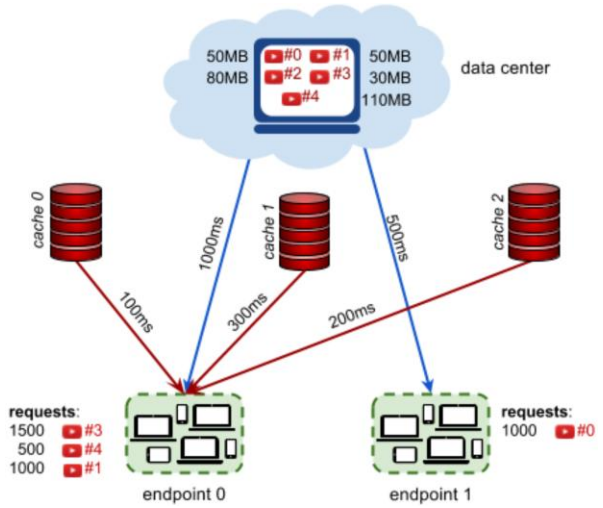
$O(C)$

$O(R \cdot C)$

best_densité = 45 000

best_move = (3, 0)

1 350 000/30 = 45 000



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	1 350 000	1 050 000	1 200 000
Vidéo 4	0	0	0

Cache 0 **0/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

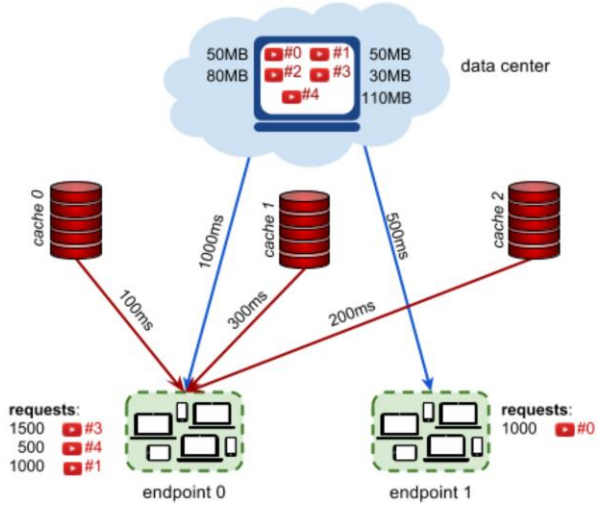
Greedy(Instance) :

```
Pour v dans V :                                   O(V)
    Pour c dans C :                               O(C)
        gain[v][c] = calcul_gain(v, c)            O(R · C)
Tant qu'il y a de la place :
    Pour v dans V :
        Pour c dans C :
            Si gain[v][c] / taille(v) > best_densité :
                best_move = (v, c)
    Place(v, c)
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```

best_densité = 45 000

best_move = (3, 0)

800 000/50 = 16 000



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0

Cache 0 Vidéo 3 **30/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

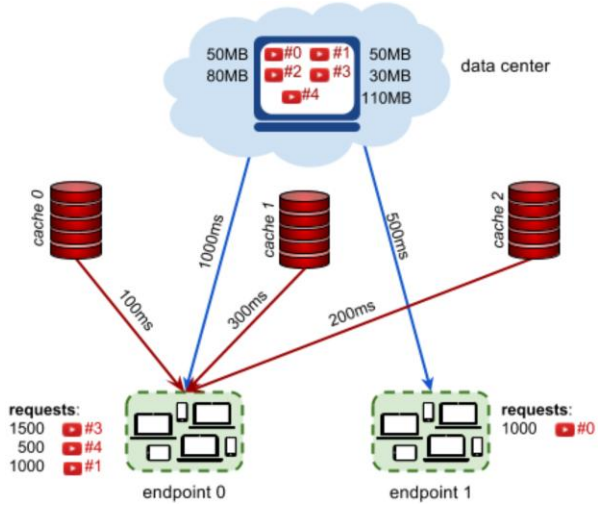
Greedy(Instance) :

- Pour v dans V : $O(V)$
  - Pour c dans C : $O(C)$
    - gain[v][c] = calcul_gain(v, c) $O(R \cdot C)$
- Tant qu'il y a de la place :
  - Pour v dans V :
    - Pour c dans C :
      - Si gain[v][c] / taille(v) > best_densité :
        - best_move = (v, c)
  - Place(v, c)
  - Pour c dans C :
    - gain[v][c] = calcul_gain(v, c)

best_densité = 18 000

best_move = (1, 0)

900 000/50 = 18 000



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	900 000	700 000	800 000
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0

Cache 0 Vidéo 3 **30/100MB**

Cache 1 **0/100MB**

Cache 2 **0/100MB**

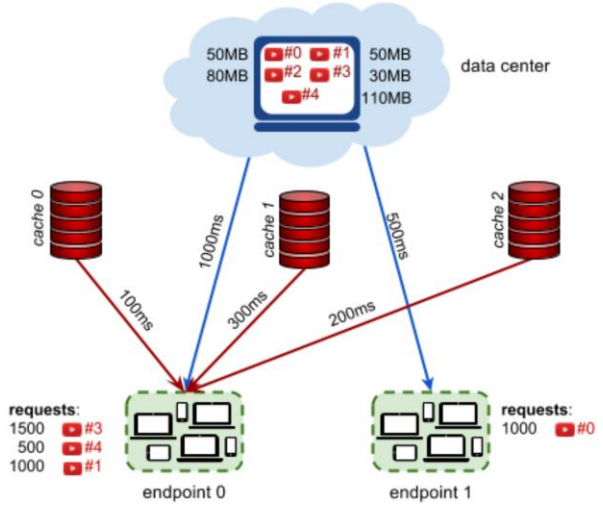
Greedy(Instance) :

```
Pour v dans V :                                   O(V)
    Pour c dans C :                               O(C)
        gain[v][c] = calcul_gain(v, c)            O(R · C)
Tant qu'il y a de la place :
    Pour v dans V :
        Pour c dans C :
            Si gain[v][c] / taille(v) > best_densité :
                best_move = (v, c)
    Place(v, c)
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```

best_densité = 18 000

best_move = (1, 0)

900 000/50 = 18 000



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	0	0	0
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0

Cache 0 Vidéo 3 | Vidéo 1 — **80/100MB**

Cache 1 — **0/100MB**

Cache 2 — **0/100MB**

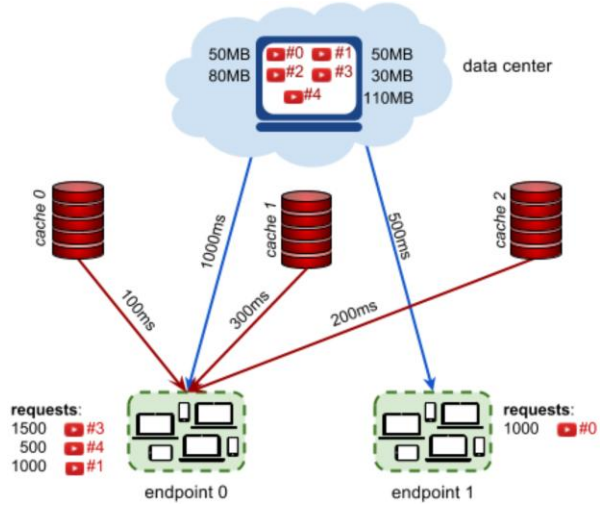
Greedy(Instance) :

```
Pour v dans V :                                   O(V)
    Pour c dans C :                               O(C)
        gain[v][c] = calcul_gain(v, c)            O(R · C)
Tant qu'il y a de la place :
    Pour v dans V :
        Pour c dans C :
            Si gain[v][c] / taille(v) > best_densité :
                best_move = (v, c)
    Place(v, c)
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```

best_densité = 0

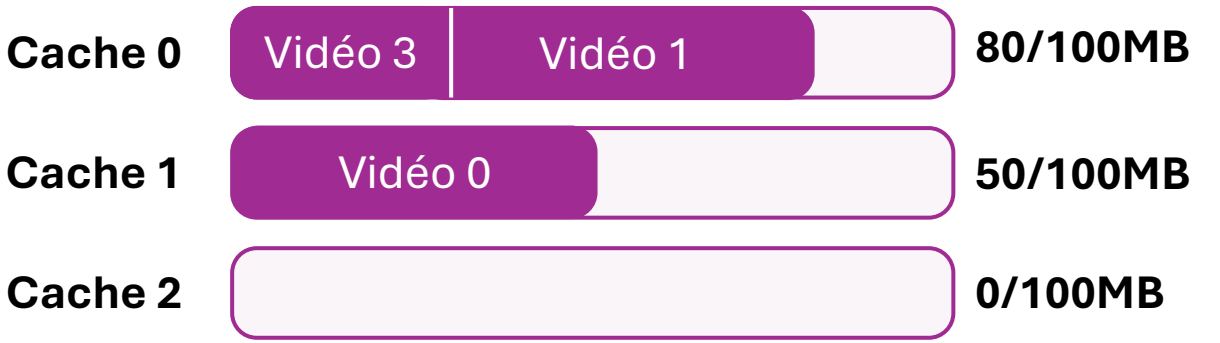
best_move = (0, 1)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	0	0	0
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0



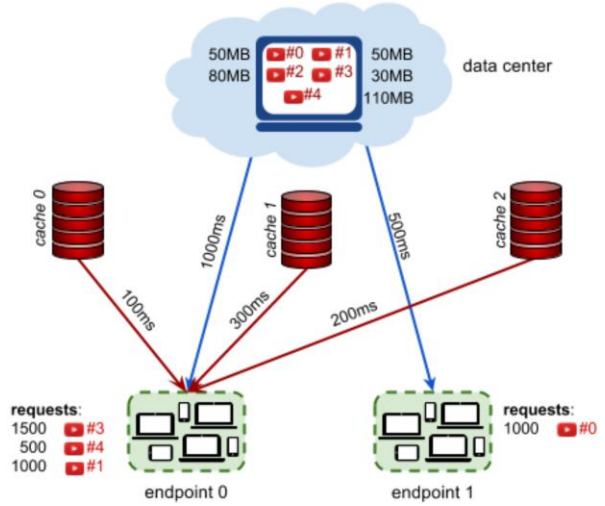
Greedy(Instance) :

- Pour v dans V : $O(V)$
  - Pour c dans C : $O(C)$
    - gain[v][c] = calcul_gain(v, c) $O(R \cdot C)$
- Tant qu'il y a de la place :
  - Pour v dans V :
    - Pour c dans C :
      - Si gain[v][c] / taille(v) > best_densité :
        - best_move = (v, c)
  - Place(v, c)
  - Pour c dans C :
    - gain[v][c] = calcul_gain(v, c)

best_densité = 0

best_move = (0, 2)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	0	0	0
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0

Cache 0 Vidéo 3 | Vidéo 1 — **80/100MB**

Cache 1 Vidéo 0 — **50/100MB**

Cache 2 Vidéo 0 — **50/100MB**

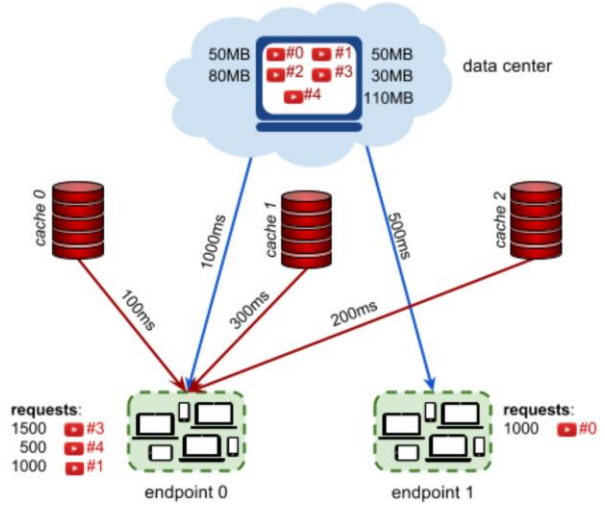
Greedy(Instance) :

```
Pour v dans V :                                   O(V)
    Pour c dans C :                               O(C)
        gain[v][c] = calcul_gain(v, c)            O(R · C)
Tant qu'il y a de la place :
    Pour v dans V :
        Pour c dans C :
            Si gain[v][c] / taille(v) > best_densité :
                best_move = (v, c)
    Place(v, c)
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```

best_densité = 0

best_move = (1, 1)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	0	0	0
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0

Cache 0 Vidéo 3 | Vidéo 1 — **80/100MB**

Cache 1 Vidéo 0 | Vidéo 1 — **100/100MB**

Cache 2 Vidéo 0 — **50/100MB**

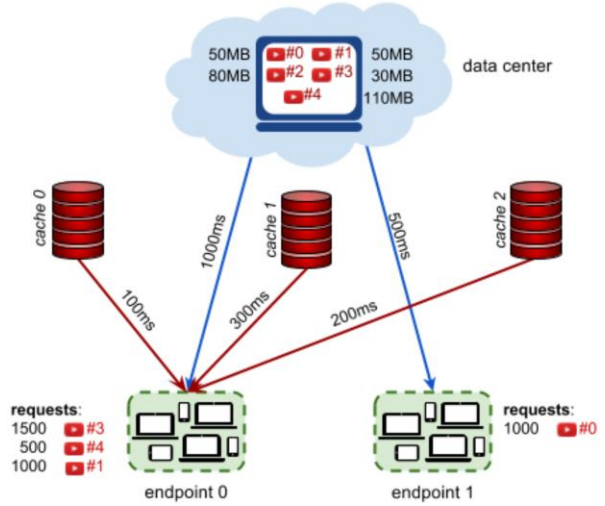
Greedy(Instance) :

```
Pour v dans V :                                   O(V)
    Pour c dans C :                               O(C)
        gain[v][c] = calcul_gain(v, c)            O(R · C)
Tant qu'il y a de la place :
    Pour v dans V :
        Pour c dans C :
            Si gain[v][c] / taille(v) > best_densité :
                best_move = (v, c)
    Place(v, c)
    Pour c dans C :
        gain[v][c] = calcul_gain(v, c)
```

best_densité = 0

best_move = (1, 2)

0/50 = 0



	Taille
Vidéo 0	50
Vidéo 1	50
Vidéo 2	80
Vidéo 3	30
Vidéo 4	110

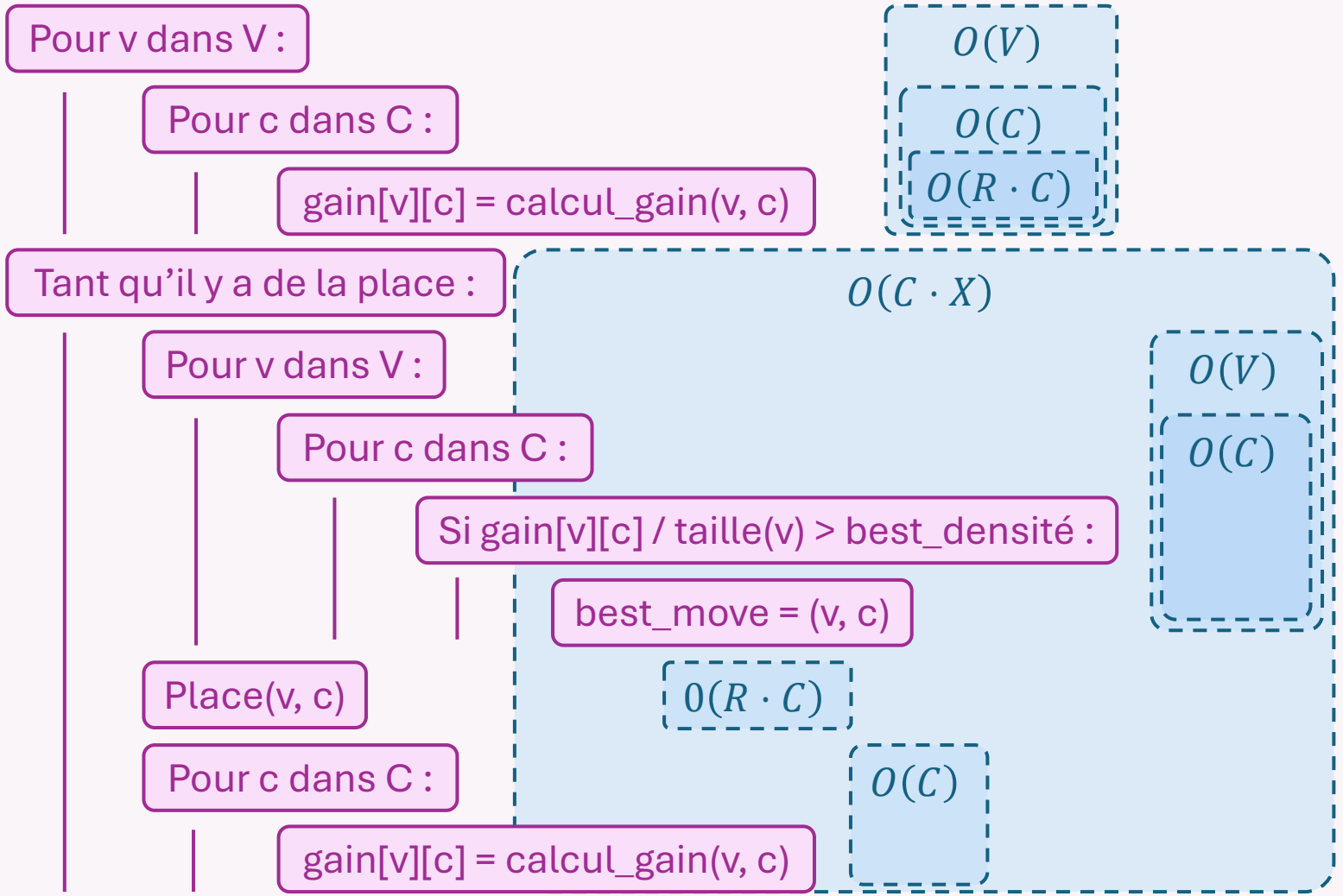
	Cache 0	Cache 1	Cache 2
Vidéo 0	0	0	0
Vidéo 1	0	0	0
Vidéo 2	0	0	0
Vidéo 3	0	0	0
Vidéo 4	0	0	0

Cache 0 Vidéo 3 | Vidéo 1 — **80/100MB**

Cache 1 Vidéo 0 | Vidéo 1 — **100/100MB**

Cache 2 Vidéo 0 | Vidéo 1 — **100/100MB**

Greedy(Instance) :



Complexité cubique

$$O\left(C^2 \cdot V \cdot R + C \cdot X(C \cdot V + R \cdot C + C)\right)$$

$$\downarrow$$

$$\boldsymbol{O\left(C^2\left(V \cdot R + X(V + R)\right)\right)}$$

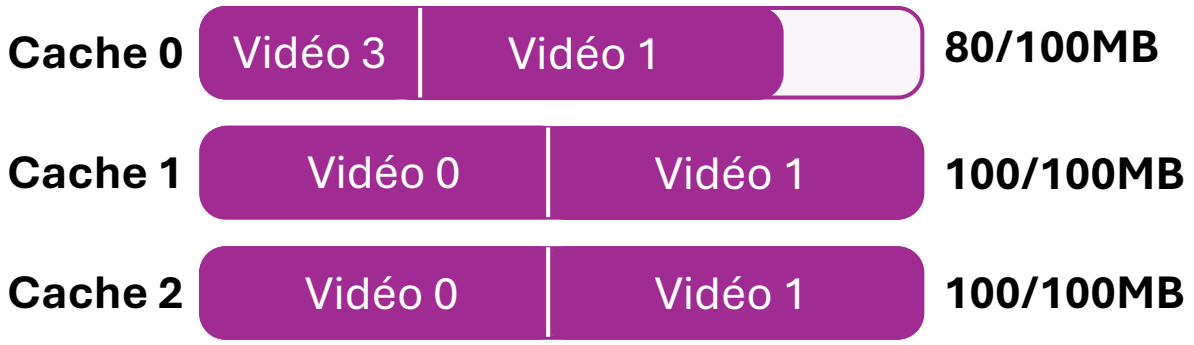
- V : Vidéos
- C : Caches
- E : Endpoints
- R : Requêtes

Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance)

Pour i de 0 à $\lambda \cdot C$:

- Cache = Random cache
- Pour vidéo dans Vidéos:
  - Retire la vidéo du cache

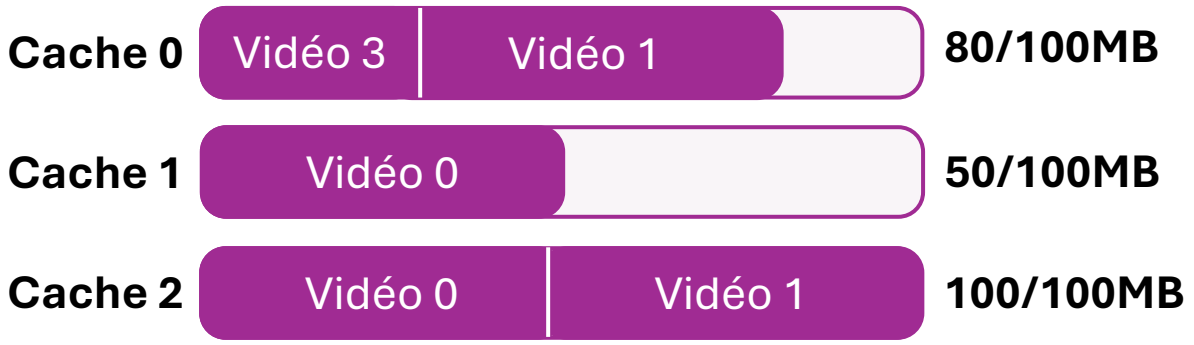


Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance)

Pour i de 0 à $\lambda \cdot C$:

- Cache = Random cache
- Pour vidéo dans Vidéos:
  - Retire la vidéo du cache



Ruin & Recreate(Instance, $\lambda = 33\%$) :

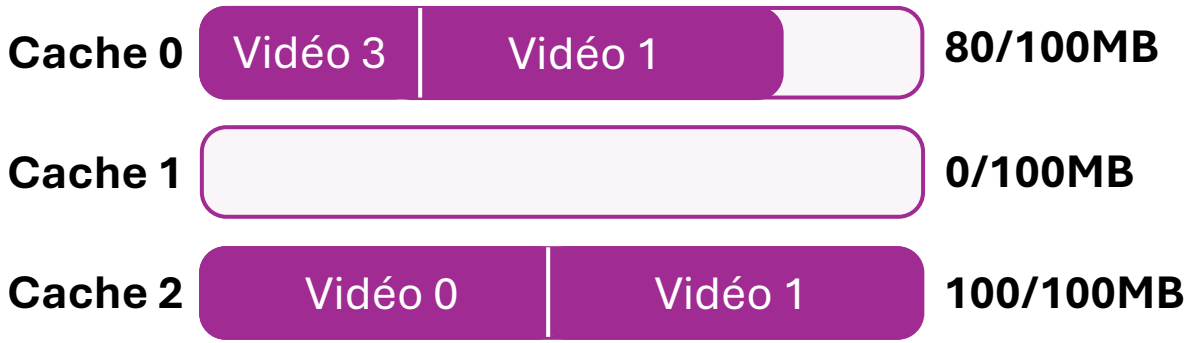
current_instance = Copy(instance)

Pour i de 0 à $\lambda \cdot C$:

    Cache = Random cache

    Pour vidéo dans Vidéos:

        Retire la vidéo du cache

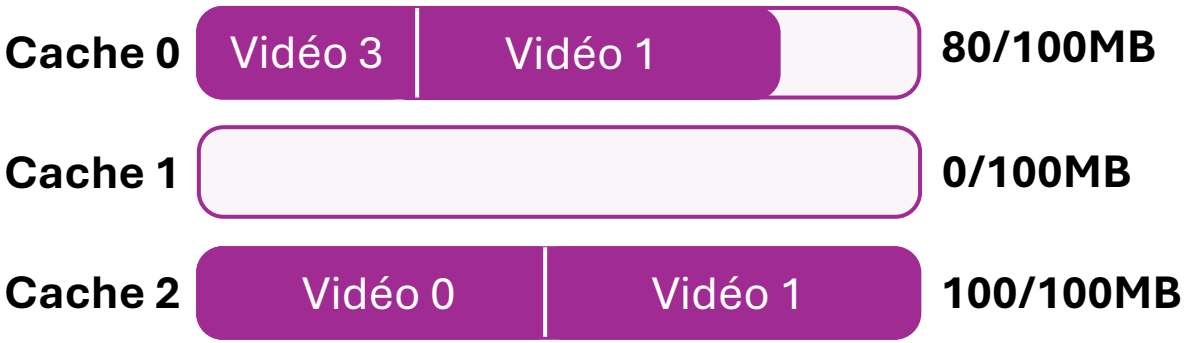


Ruin & Recreate(Instance, $\lambda = 33\%$) :

current_instance = Copy(instance) — $O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$: — $O(\lambda \cdot C)$

- Cache = Random cache
- Pour vidéo dans Vidéos: — $O(V)$
  - Retire la vidéo du cache



Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance)

$O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$:

$O(\lambda \cdot C)$

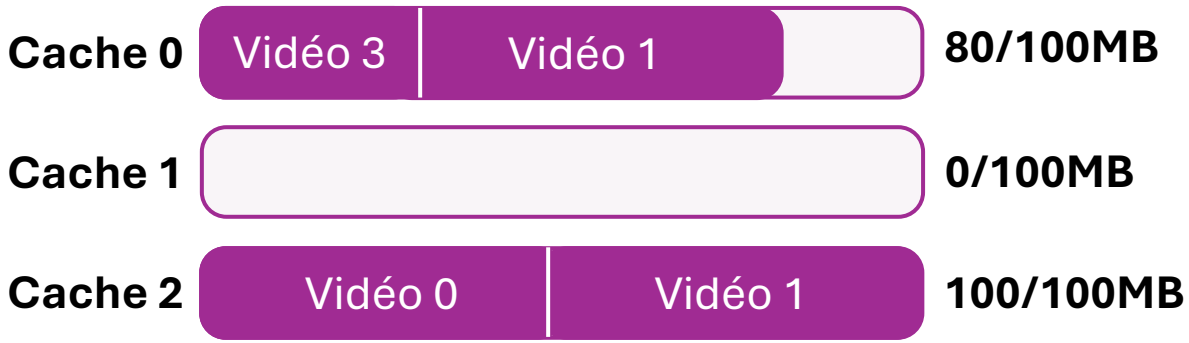
Cache = Random cache

Pour vidéo dans Vidéos:

$O(V)$

Retire la vidéo du cache

Greedy(current_instance)



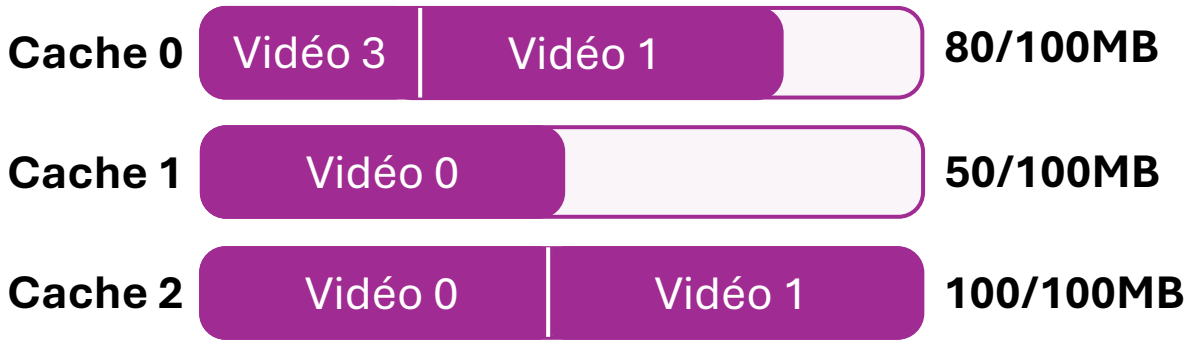
Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance) — $O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$: — $O(\lambda \cdot C)$

- Cache = Random cache
- Pour vidéo dans Vidéos: — $O(V)$
  - Retire la vidéo du cache

Greedy(current_instance)



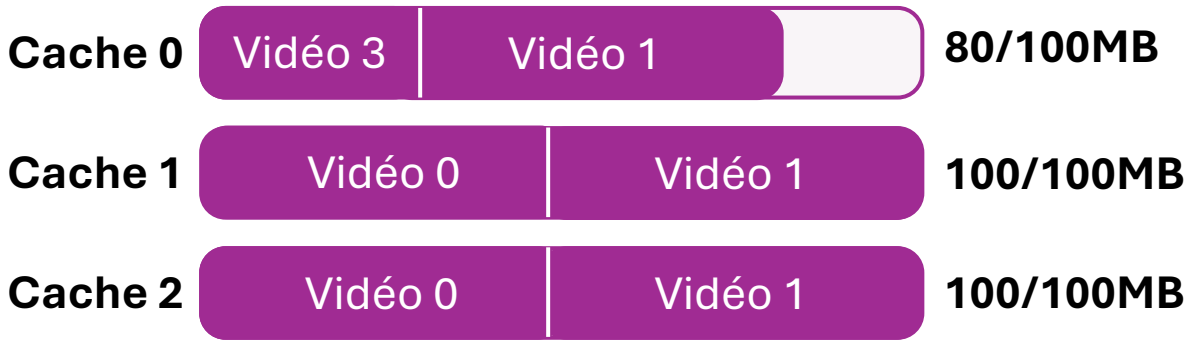
Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance) — $O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$: — $O(\lambda \cdot C)$

- Cache = Random cache
- Pour vidéo dans Vidéos: — $O(V)$
  - Retire la vidéo du cache

Greedy(current_instance)



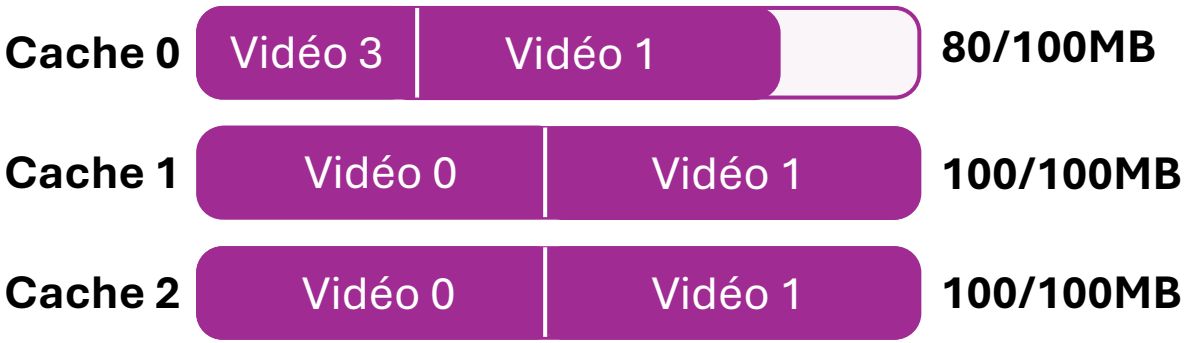
Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance) — $O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$: — $O(\lambda \cdot C)$

- Cache = Random cache
- Pour vidéo dans Vidéos: — $O(V)$
  - Retire la vidéo du cache

Greedy(current_instance) — $O\left(\lambda \cdot C^2(V \cdot R + X(V + R))\right)$



Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance) — $O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$: — $O(\lambda \cdot C)$

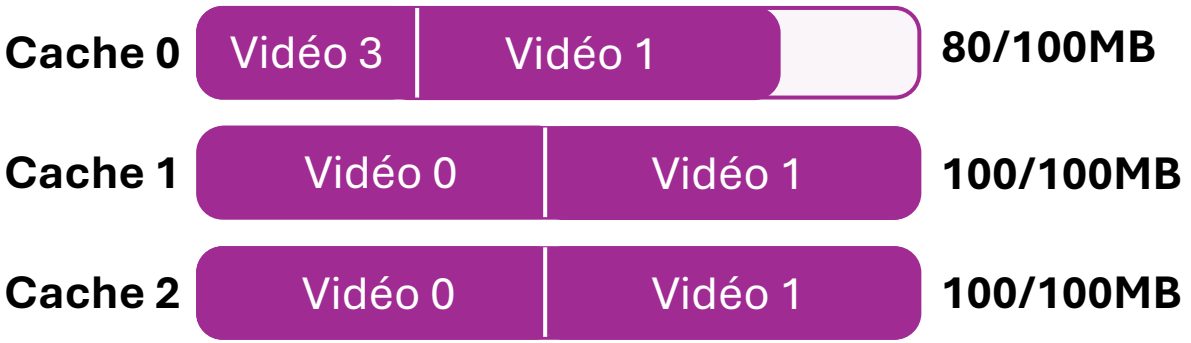
- Cache = Random cache
- Pour vidéo dans Vidéos: — $O(V)$
  - Retire la vidéo du cache

Greedy(current_instance) — $O\left(\lambda \cdot C^2(V \cdot R + X(V + R))\right)$

Si current_instance.score > instance.score : — $O(1)$

- Swap(instance, current_instance)

Free(current_instance) — $O(V + R + E \cdot C)$



Ruin & Recreate(Instance, λ = 33%) :

current_instance = Copy(instance) — $O(R + C(E + V))$

Pour i de 0 à $\lambda \cdot C$: — $O(\lambda \cdot C)$

- Cache = Random cache
- Pour vidéo dans Vidéos: — $O(V)$
  - Retire la vidéo du cache

Greedy(current_instance) — $O\left(\lambda \cdot C^2(V \cdot R + X(V + R))\right)$

Si current_instance.score > instance.score : — $O(1)$

- Swap(instance, current_instance)

Free(current_instance) — $O(\mathrm{V} + R + E \cdot \mathrm{C})$

Complexité cubique

$$O\left(R + C(E + V) + \lambda \cdot C \cdot V + \lambda \cdot C^2(V \cdot R + X(V + R))\right)$$

$$\downarrow$$

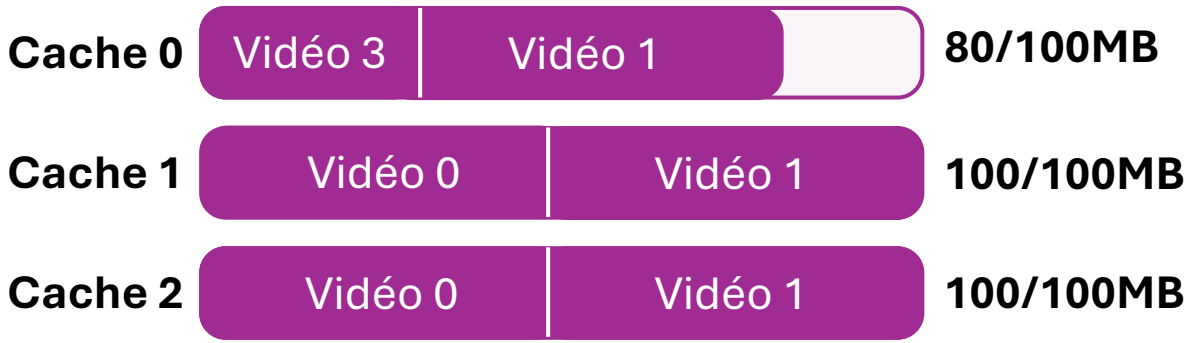
$$\boldsymbol{O\left(\lambda \cdot C^2(V \cdot R + X(V + R))\right)}$$

- V : Vidéos
- C : Caches
- E : Endpoints
- X: Capacité des caches
- R : Requêtes
- λ : taux de destruction de cache

Knapsack(Instance) :

- Initialise_knapsack()
- Tant que le score s’améliore :
  - Pour c dans Caches :
    - Pour v dans Vidéos :
      - gain[v] = calcul_gain(v, c)

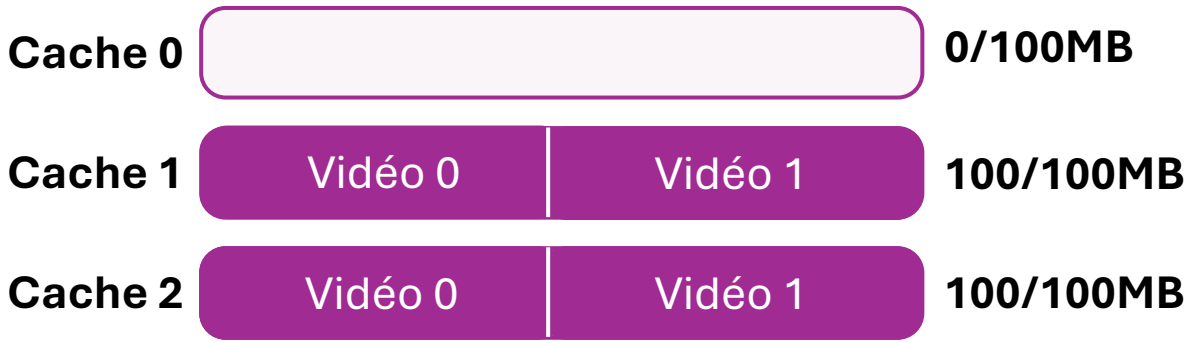
	Cache 0
Vidéo 0	
Vidéo 1	
Vidéo 2	
Vidéo 3	



Knapsack(Instance) :

- Initialise_knapsack()
- Tant que le score s'améliore :
  - Pour c dans Caches :
    - Pour v dans Vidéos :
      - gain[v] = calcul_gain(v, c)

	Cache 0
Vidéo 0	0
Vidéo 1	900 000
Vidéo 2	0
Vidéo 3	1 350 000



Knapsack(Instance) :

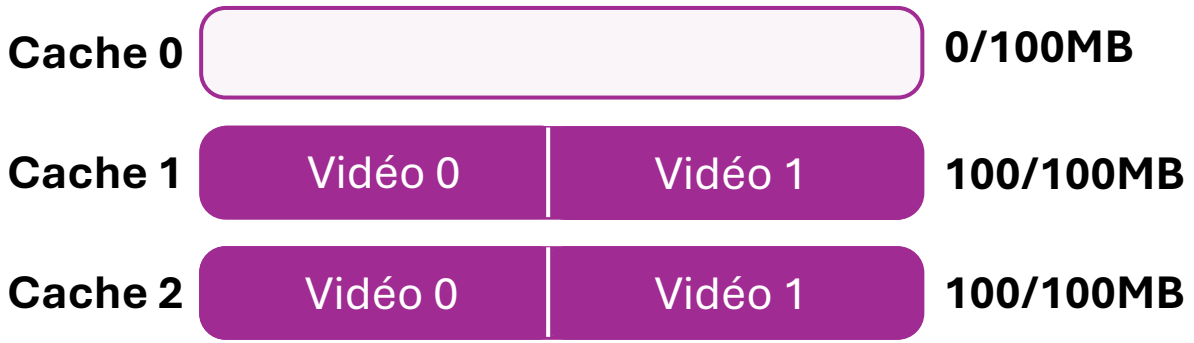
Initialise_knapsack()

Tant que le score s'améliore :

- Pour c dans Caches :
  - Pour v dans Vidéos :
    - gain[v] = calcul_gain(v, c)
  - Pour i de 0 à V :
    - Pour capa dans (Vidéos.taille U Capa_cache) :
      - Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
      - Sinon : Opt[capa][i] = Max(Opt[capa][i-1], Opt[capa-Vidéos[i].taille][i-1] + gain[v])
    - Mise à jour du cache

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0				
Vidéo 1	0				
Vidéo 2	0				
Vidéo 3	0				

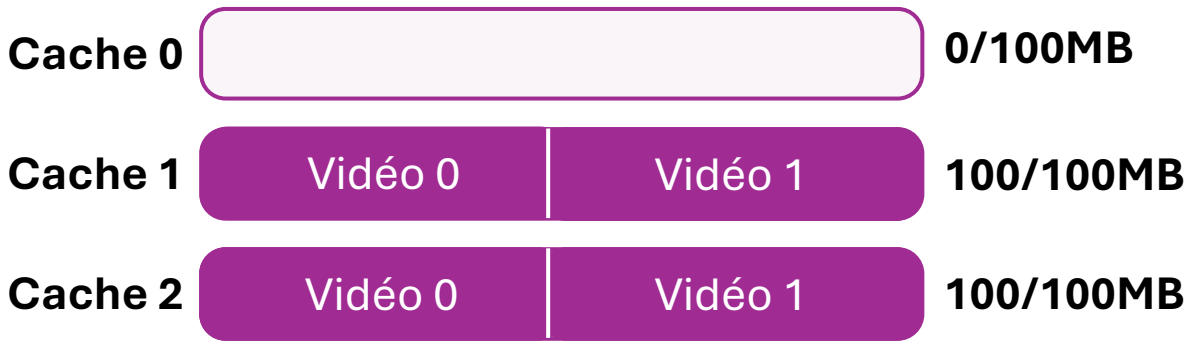


Knapsack(Instance) :

- Initialise_knapsack()
- Tant que le score s'améliore :
  - Pour c dans Caches :
    - Pour v dans Vidéos :
      - gain[v] = calcul_gain(v, c)
    - Pour i de 0 à V :
      - Pour capa dans (Vidéos.taille U Capa_cache) :
        - Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
        - Sinon : Opt[capa][i] = Max(Opt[capa][i-1], Opt[capa-Vidéos[i].taille][i-1] + gain[v])
      - Mise à jour du cache

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0			
Vidéo 1	0				
Vidéo 2	0				
Vidéo 3	0				

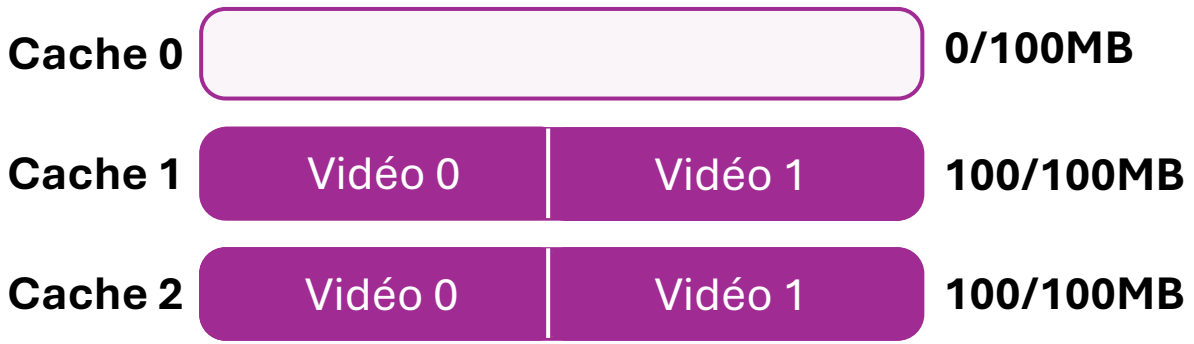


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0		
Vidéo 1	0				
Vidéo 2	0				
Vidéo 3	0				

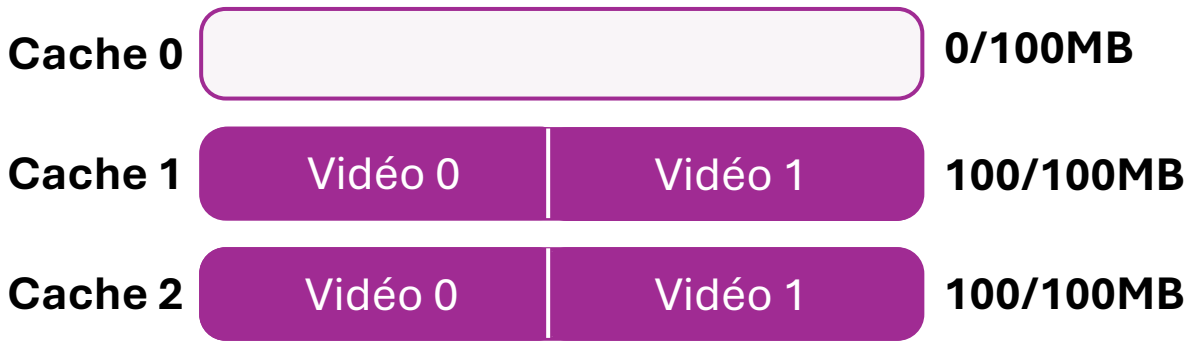


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	
Vidéo 1	0				
Vidéo 2	0				
Vidéo 3	0				



Knapsack(Instance) :

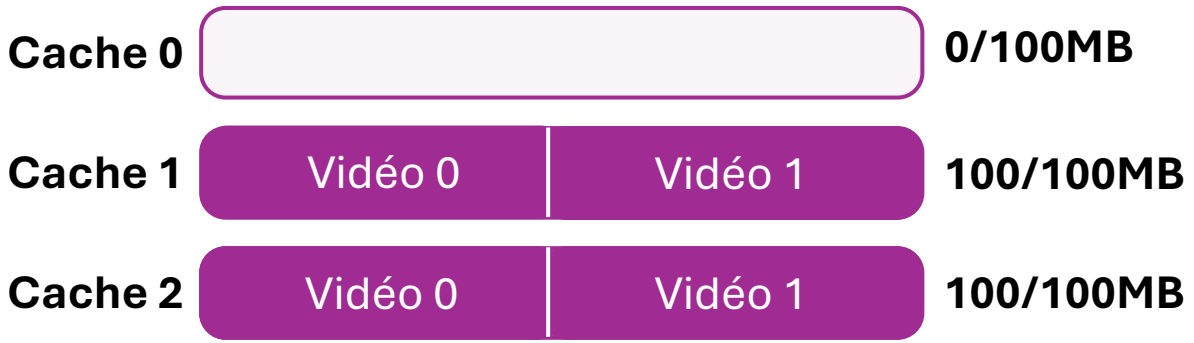
Initialise_knapsack()

Tant que le score s'améliore :

- Pour c dans Caches :
  - Pour v dans Vidéos :
    - gain[v] = calcul_gain(v, c)
  - Pour i de 0 à V :
    - Pour capa dans (Vidéos.taille U Capa_cache) :
      - Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
      - Sinon : Opt[capa][i] = Max(Opt[capa][i-1], Opt[capa-Vidéos[i].taille][i-1] + gain[v])
  - Mise à jour du cache

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0				
Vidéo 2	0				
Vidéo 3	0				

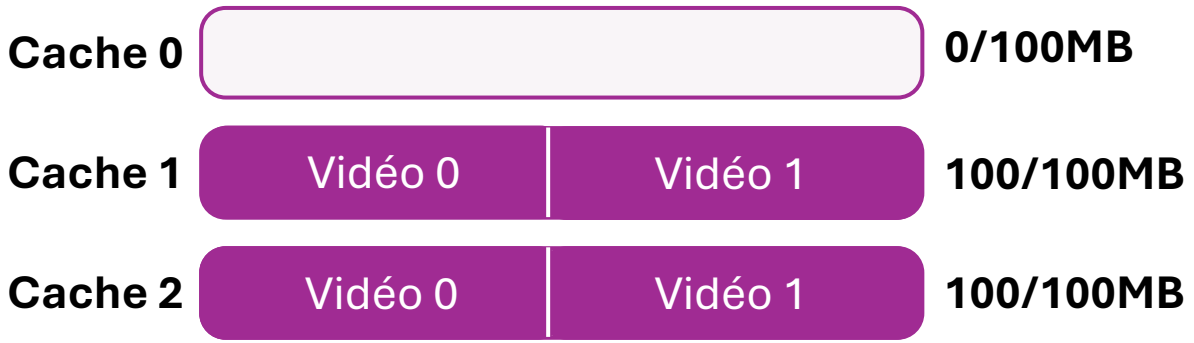


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0			
Vidéo 2	0				
Vidéo 3	0				

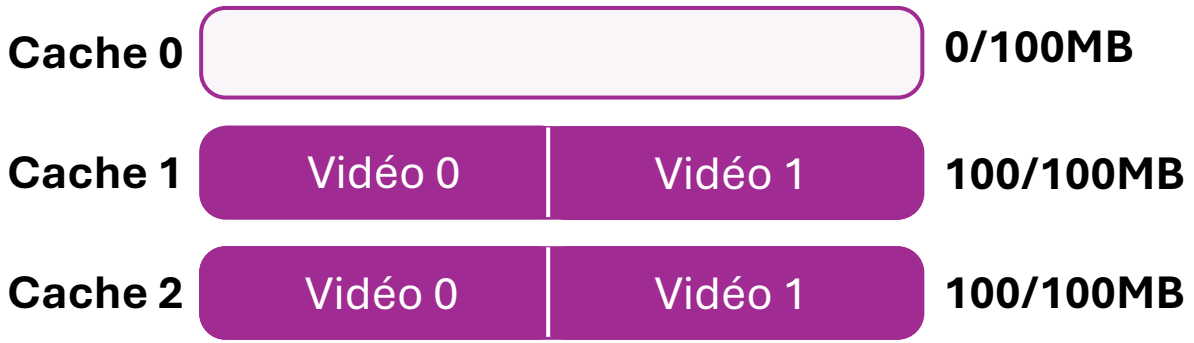


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000		
Vidéo 2	0				
Vidéo 3	0				

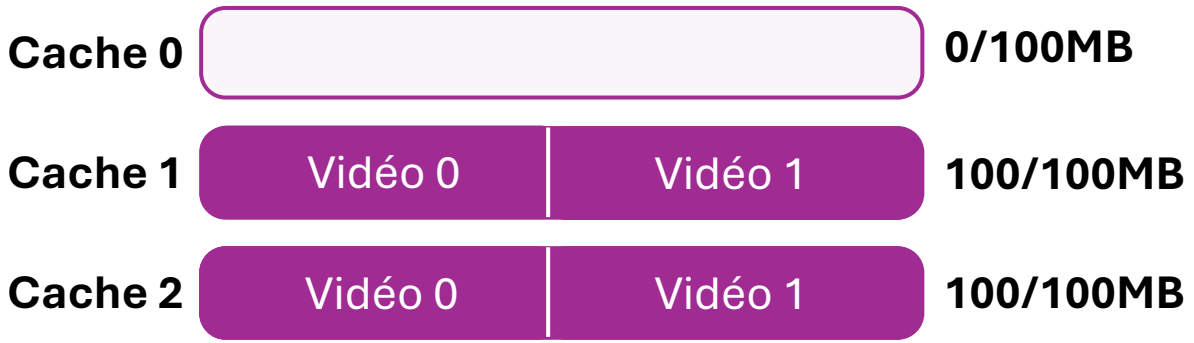


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	
Vidéo 2	0				
Vidéo 3	0				

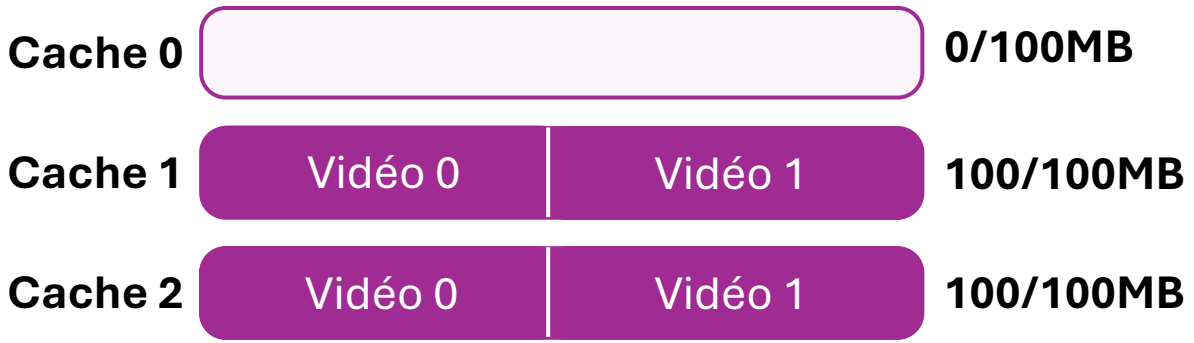


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0				
Vidéo 3	0				

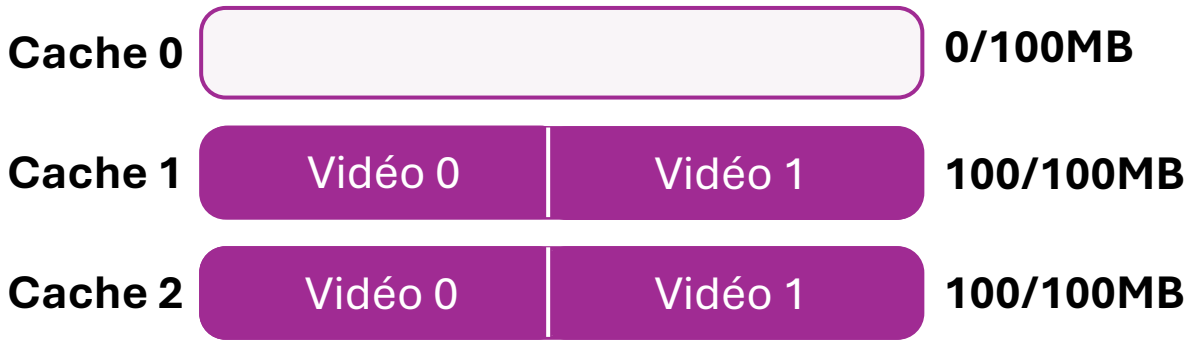


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0			
Vidéo 3	0				

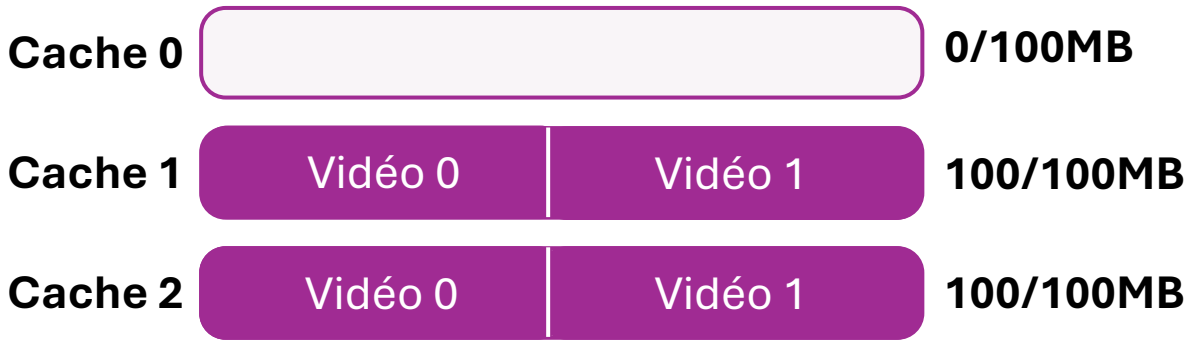


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000		
Vidéo 3	0				

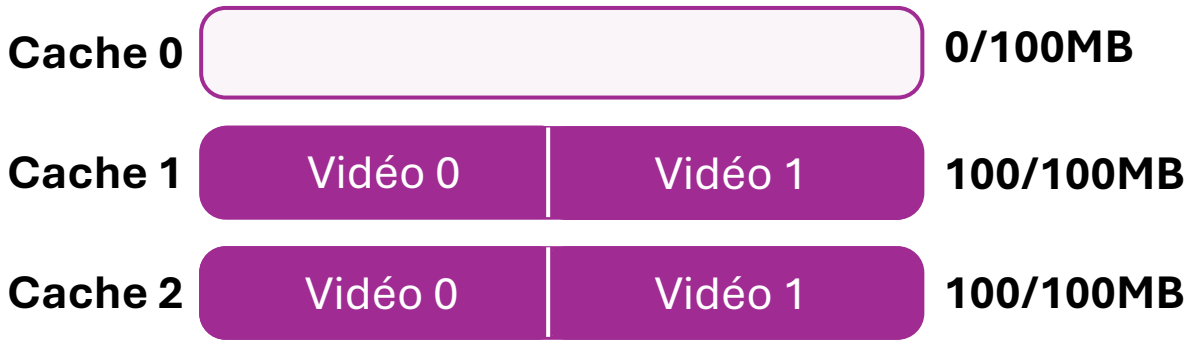


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	
Vidéo 3	0				

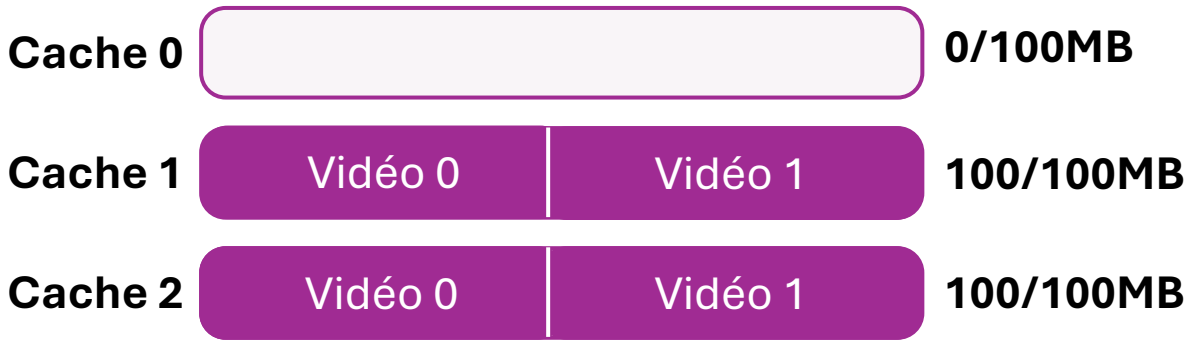


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	900 000
Vidéo 3	0				

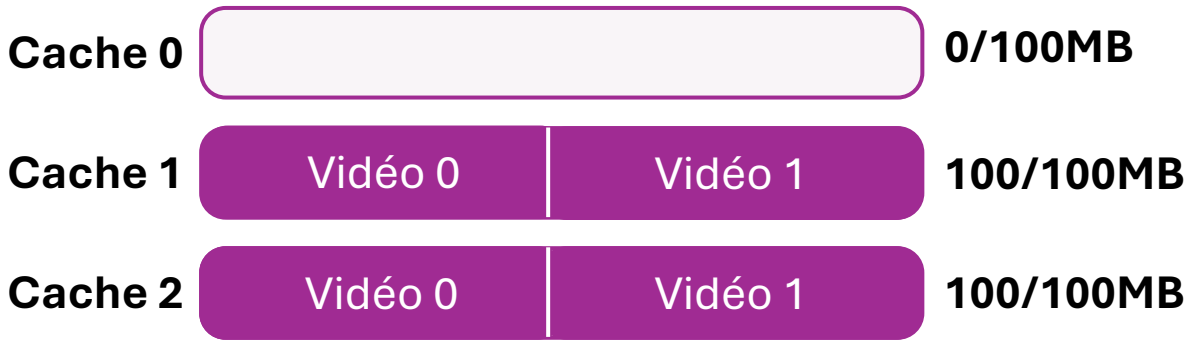


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	900 000
Vidéo 3	0	1 350 000			

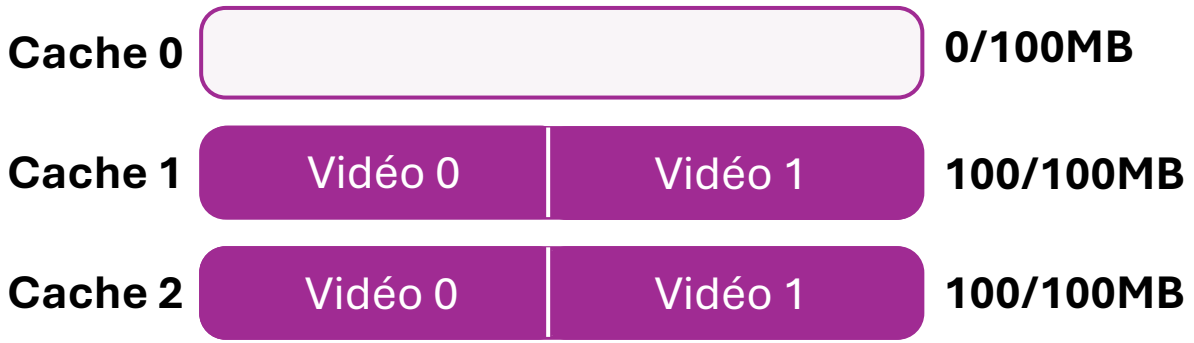


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	900 000
Vidéo 3	0	1 350 000	1 350 000		

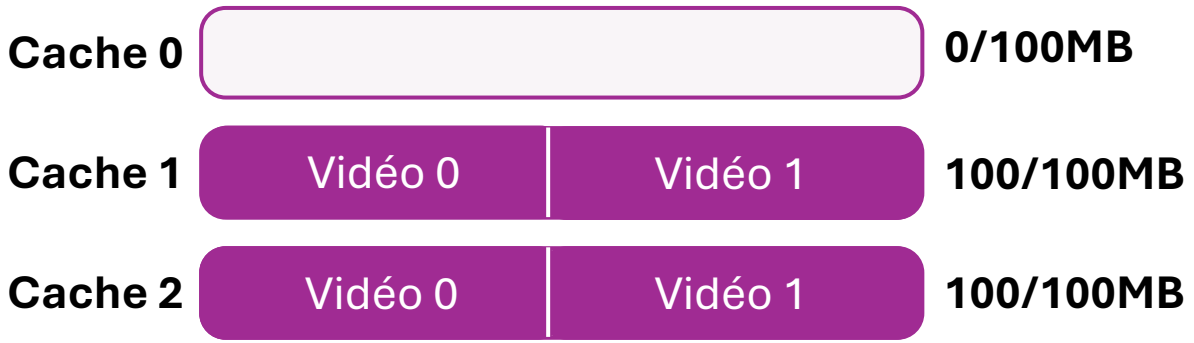


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
∅	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	900 000
Vidéo 3	0	1 350 000	1 350 000	2 250 000	

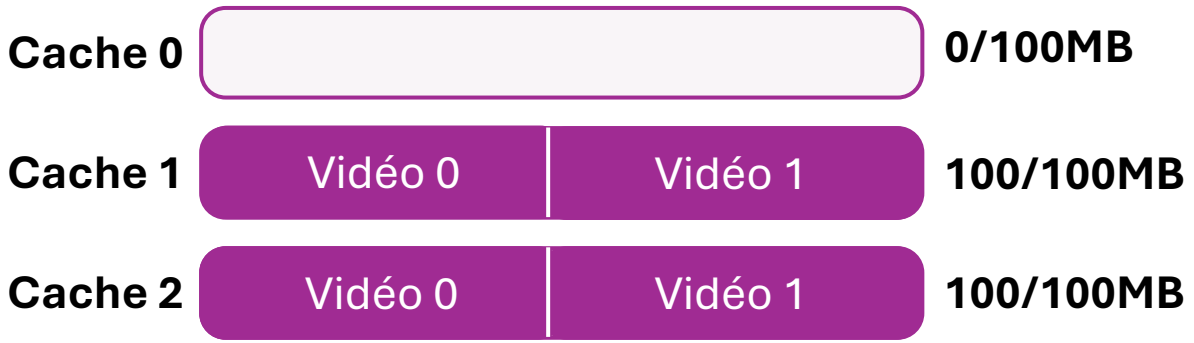


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	900 000
Vidéo 3	0	1 350 000	1 350 000	2 250 000	2 250 000

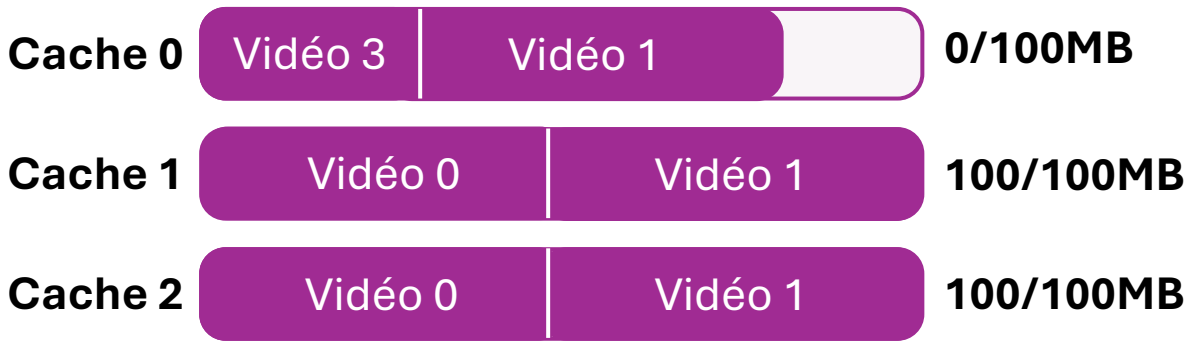


Knapsack(Instance) :

```
Initialise_knapsack()
Tant que le score s'améliore :
    Pour c dans Caches :
        Pour v dans Vidéos :
            gain[v] = calcul_gain(v, c)
        Pour i de 0 à  V :
            Pour capa dans (Vidéos.taille U Capa_cache) :
                Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
                Sinon : Opt[capa][i] = Max(Opt[capa][i-1],
                Opt[capa-Vidéos[i].taille][i-1] + gain[v])
        Mise à jour du cache
```

	Taille	Cache 0
Vidéo 0	50	0
Vidéo 1	50	900 000
Vidéo 2	80	0
Vidéo 3	30	1 350 000

	0	30	50	80	100
Ø	0	0	0	0	0
Vidéo 0	0	0	0	0	0
Vidéo 1	0	0	900 000	900 000	900 000
Vidéo 2	0	0	900 000	900 000	900 000
Vidéo 3	0	1 350 000	1 350 000	2 250 000	2 250 000



Knapsack(Instance) :

Initialise_knapsack() $O\big(R(C+V)\big)$

Tant que le score s'améliore :

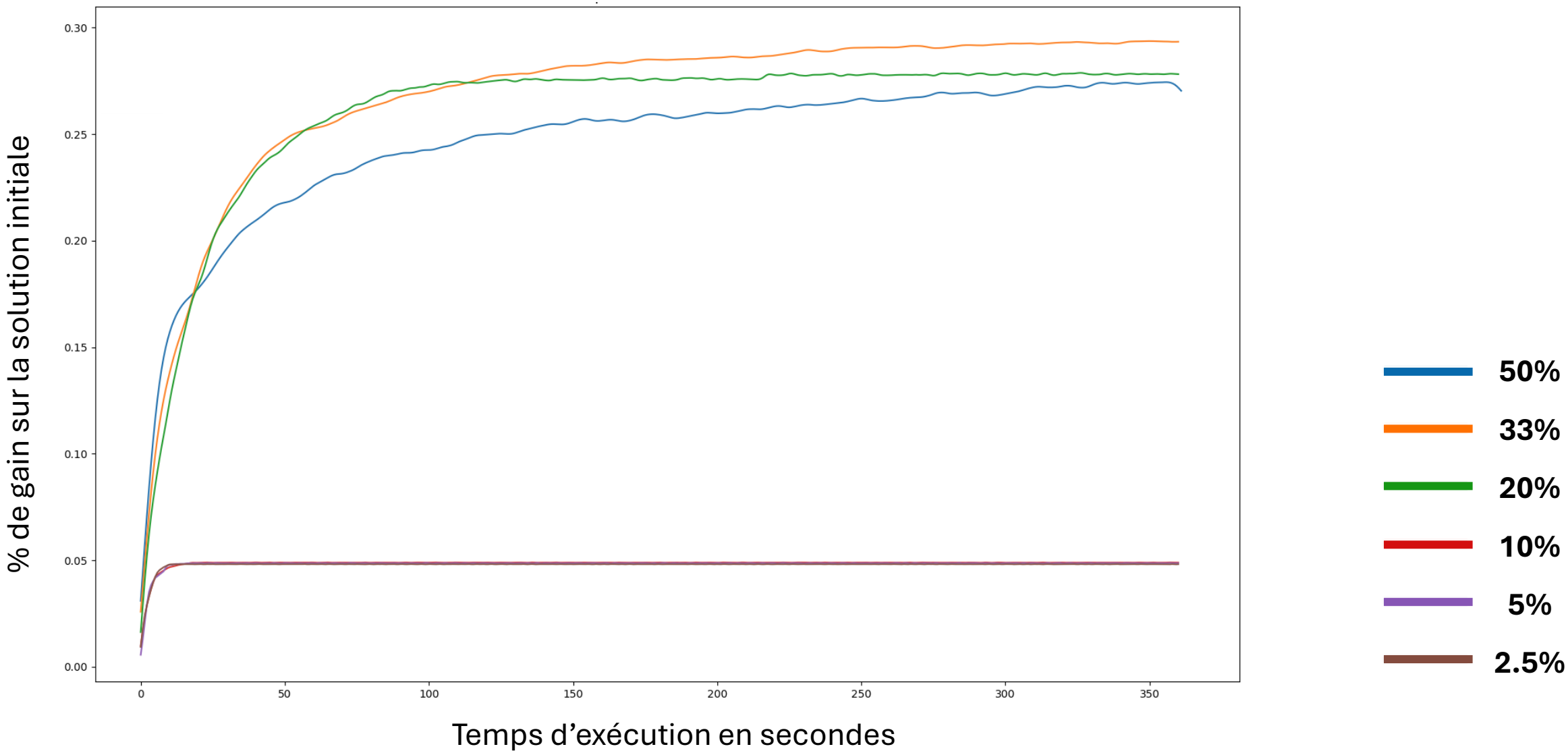
- Pour c dans Caches : $O(V)$
  - Pour v dans Vidéos : $O(C)$
    - gain[v] = calcul_gain(v, c) $O(R \cdot C)$
  - Pour i de 0 à V : $O(V^2)$
    - Pour capa dans (Vidéos.taille U Capa_cache) :
      - Si Vidéos[i].taille > capa : Opt[capa][i] = Opt[capa][i-1]
      - Sinon : Opt[capa][i] = Max(Opt[capa][i-1], Opt[capa-Vidéos[i].taille][i-1] + gain[v])
  - Mise à jour du cache $O(V)$

Complexité cubique

$$\boldsymbol{O\big(R(C+V) + C^2 \cdot V \cdot R + V^2\big)}$$

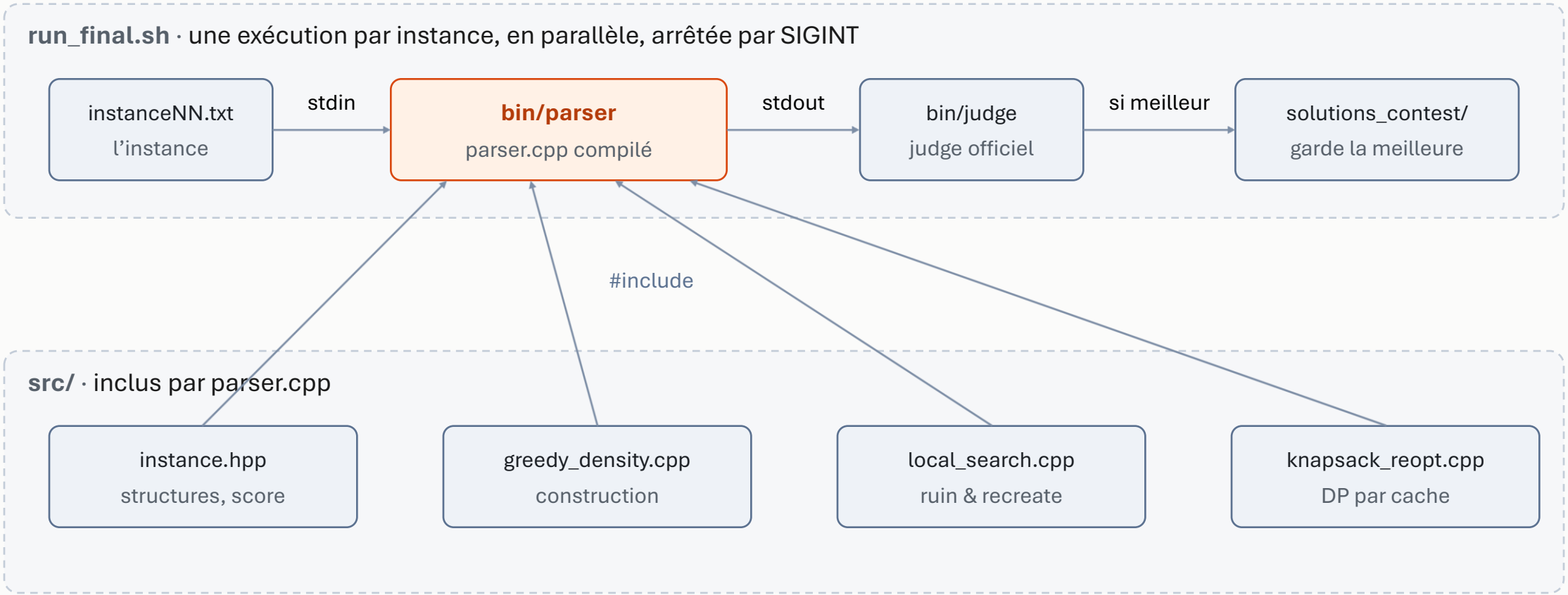
- V : Vidéos
- C : Caches
- E : Endpoints
- X: Capacité des caches
- R : Requêtes
- λ : taux de destruction de cache

Comparaison de différents λ sur instance2 (hors compétition)



Code

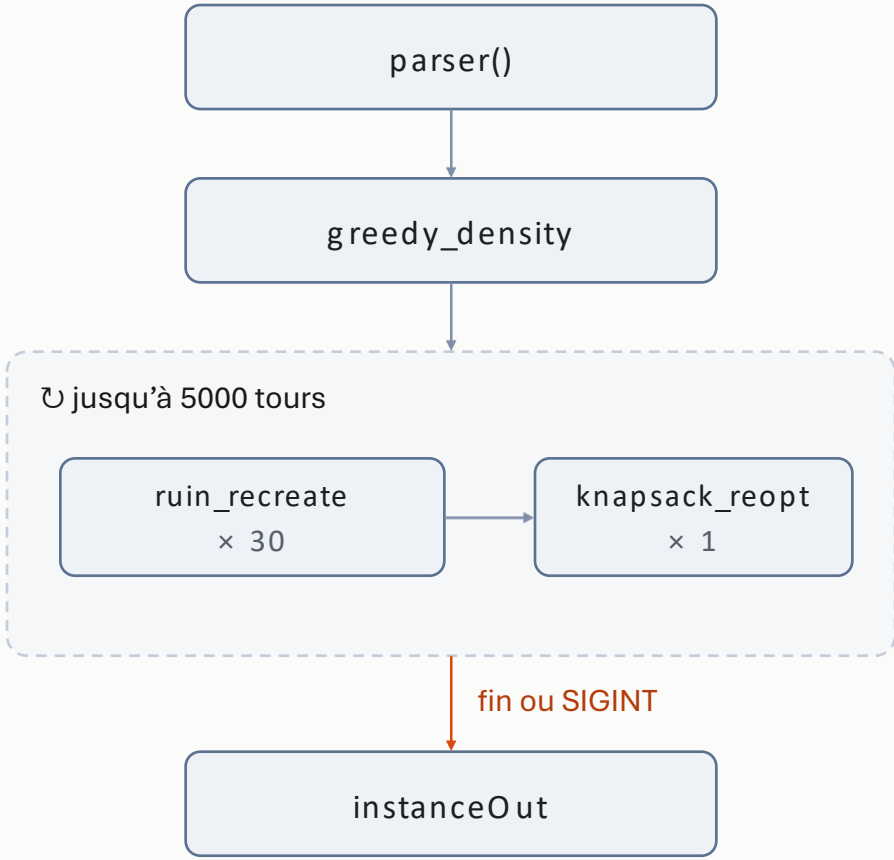
Architecture



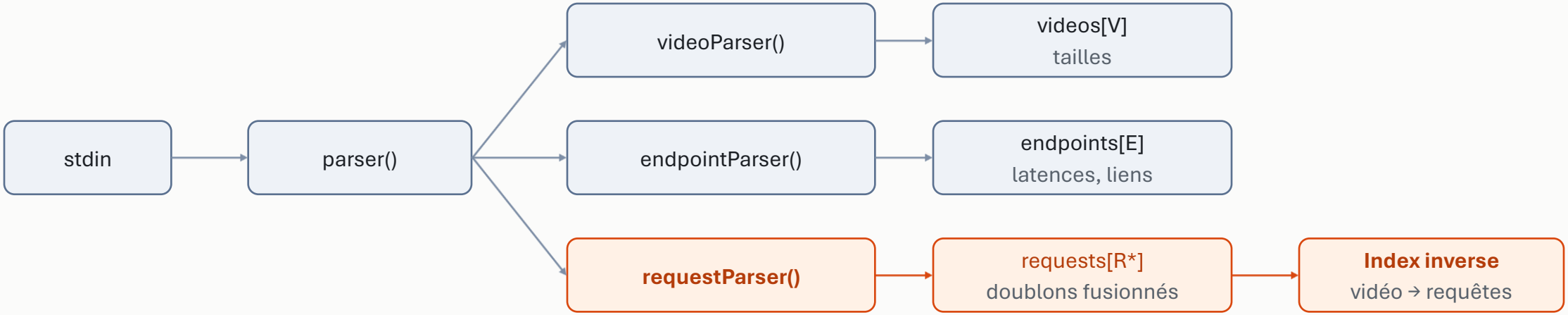
main() en quatre temps

```
InstanceData instance = parser();
std::signal(SIGINT, requestStop);
std::signal(SIGTERM, requestStop);

greedy_density(&instance);
int num_moves = std::max(1, instance.ip.C / 5);
for (int j = 0; j < 5000 && !stop_requested; j++) {
  for (int i = 0; i < 30 && !stop_requested; i++)
    ruin_recreate(&instance, 1, num_moves);
  if (stop_requested) break;
  knapsack_reopt(&instance);
}
instanceOut(&instance); // stdout
```



Lecture : fusionner et indexer




```
// requestParser(), simplifié
auto& m = requestMap[endpoint_id];
if (m.find(video_id) != m.end()) m[video_id] += count;
else { m[video_id] = count; Rprime++; }
videos[v].associated_requests.push_back(idR);
```

R → R* : une seule requête par paire (vidéo, endpoint), qui garde son gain courant.

Index inverse : déplacer une vidéo ne relit que ses requêtes.

Trois choix dans le code

```
volatile std::sig_atomic_t stop_requested = 0;
void requestStop(int sig) {
   stop_requested = 1;
   std::signal(sig, SIG_DFL);
}
```

Arrêt propre

Le mouvement en cours finit, puis la solution part sur stdout. Un 2e Ctrl-C tue le processus.

```
InstanceData current = deepCopyInstanceData(instance);
ruin(&current, num_moves, p_ruin);
greedy_density(&current, p_best);
if (current.score > best.score) std::swap(best, current);
freeInstanceData(&current);
```

Copie, essai, swap

L’essai se fait sur une copie. Garder le meilleur = échanger deux structures

```
long long score; // Σ gain ×  count
instance->score += score_diff; // delta d’un mouvement
```

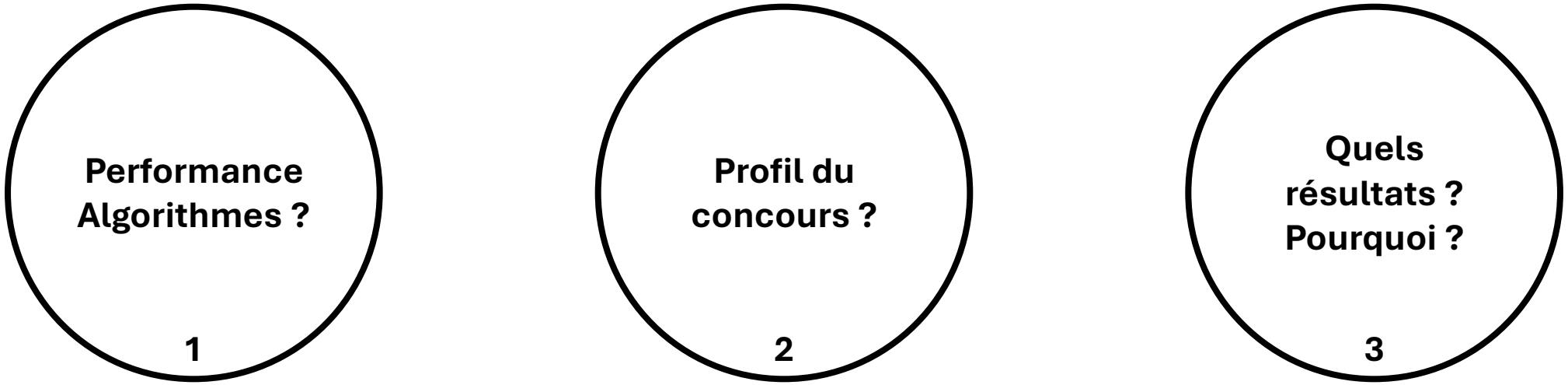
Score incrémental

Expériences

Données disponibles

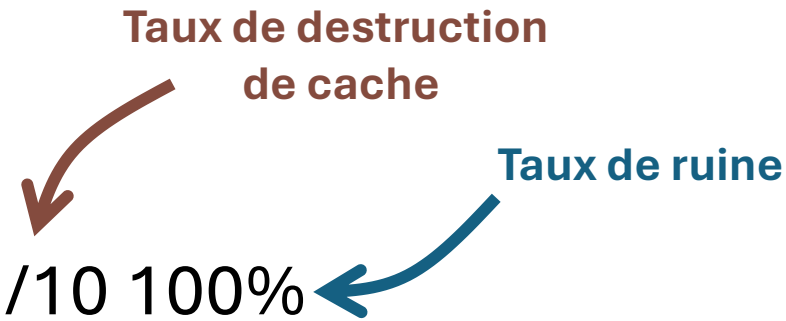
- 30 Instances non compétitives
  - Résultats Glouton
  - Meilleurs résultats tous groupes confondus
- 15 Instances compétitives
  - Résultats Compétitions + Algorithmes intermédiaires
  - Meilleurs résultats tous groupes confondus
- Présentation no. 2
  - Métriques
  - Corrélations
  - Catégorisation par ACP

Questions centrales



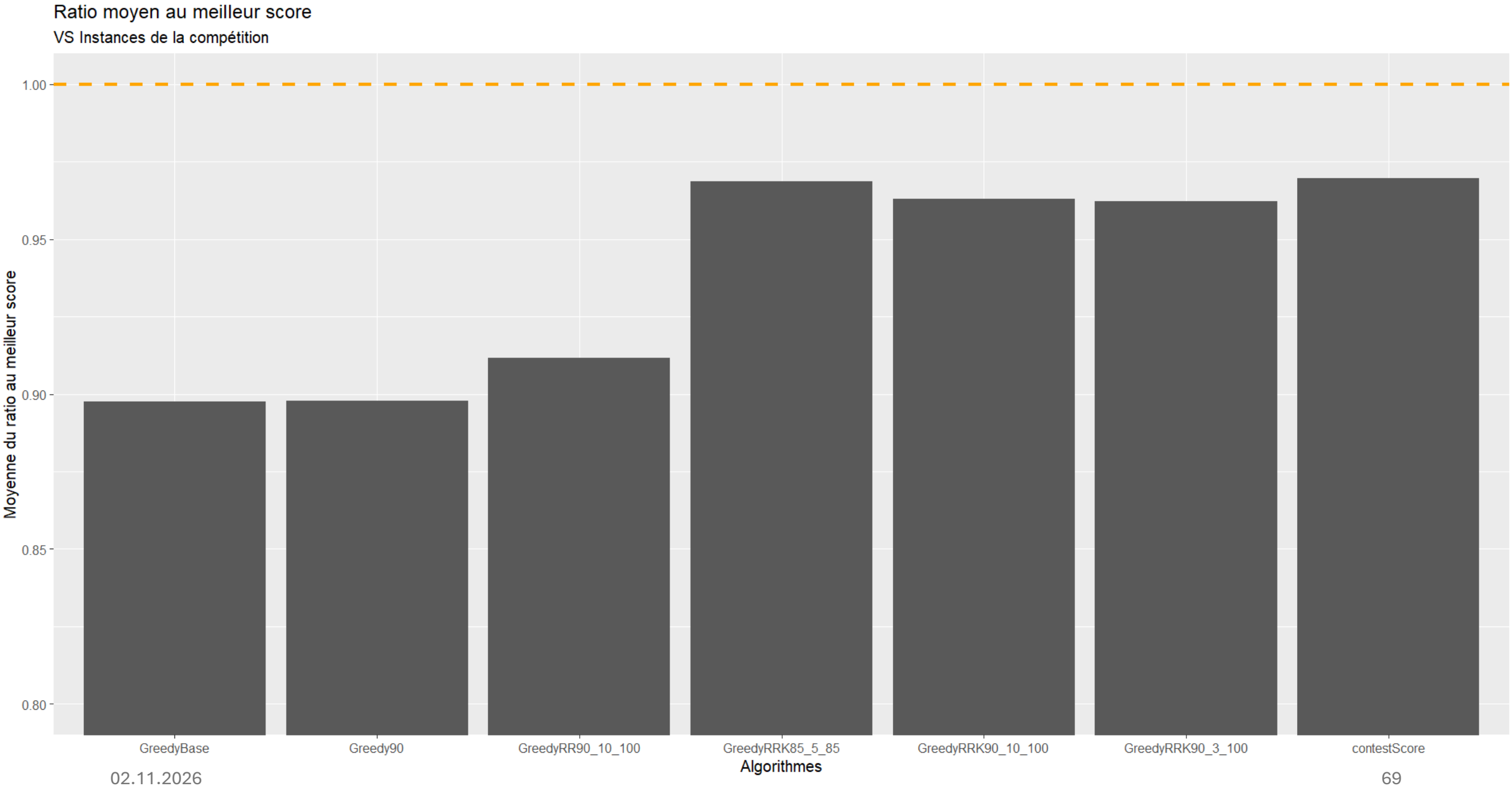
Algorithmes pour les expériences

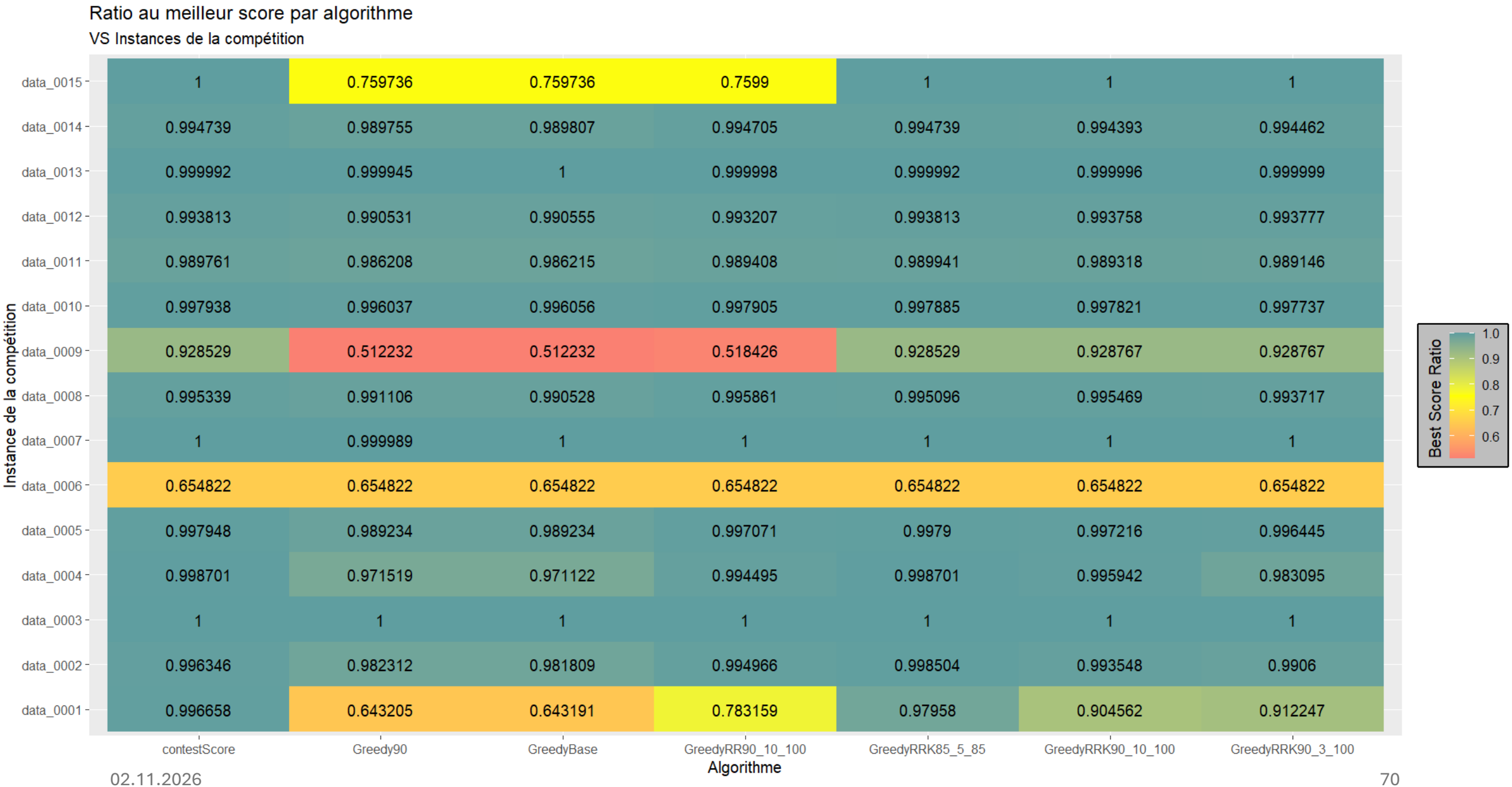
- Glouton Densité
- Glouton Stochastique (90%)
- Glouton Stochastique (90%) + R&R 1/10 100%
- Glouton Stochastique (90%) + R&R 1/10 100% + Knapsack
- Glouton Stochastique (90%) + R&R 1/3 100% + Knapsack
- Glouton Stochastique (85%) + R&R 1/5 85% + Knapsack



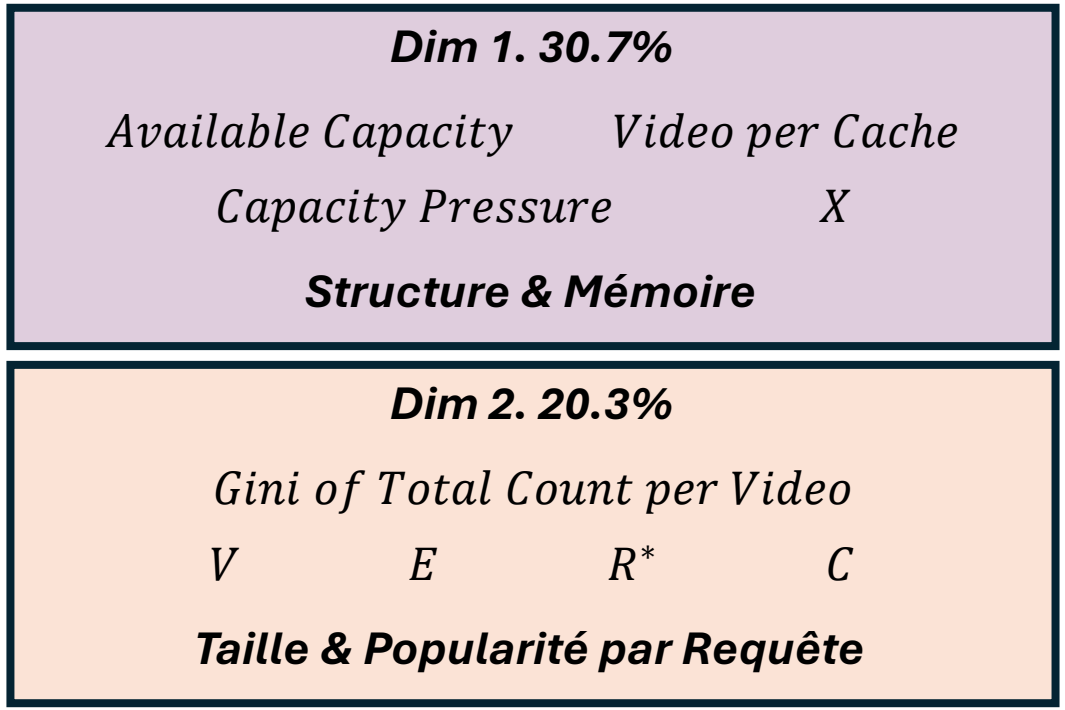
Temps Autorisé : 10 Minutes Maximum

1 Répétition → Comme pour le concours, mêmes paramètres

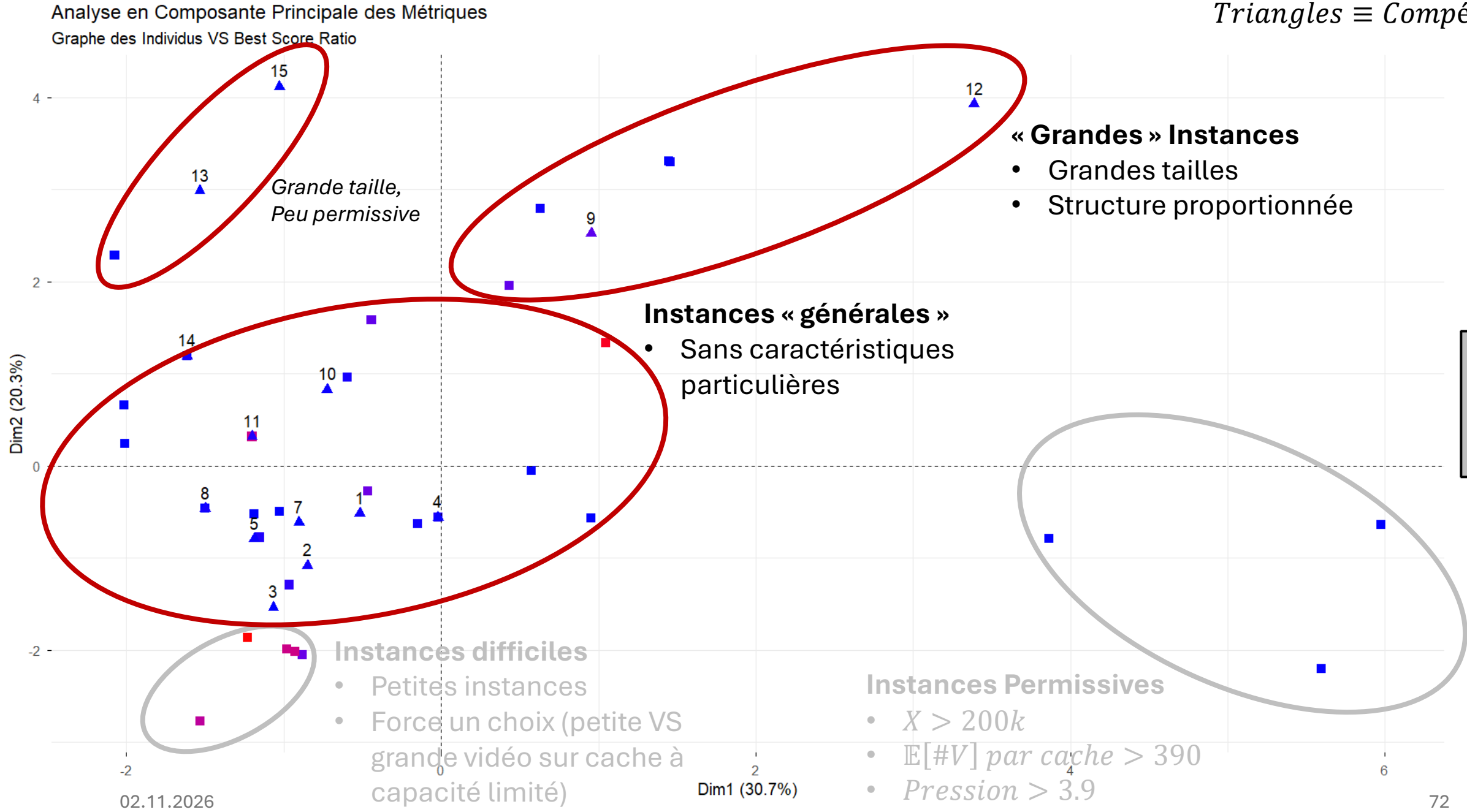

Ratio moyen au meilleur score
VS Instances de la compétition
1.00
0.95
0.90
0.85
0.80
Moyenne du ratio au meilleur score
GreedyBase
Greedy90
GreedyRR90_10_100
GreedyRRK85_5_85
GreedyRRK90_10_100
GreedyRRK90_3_100
contestScore
Algorithmes


Ratio au meilleur score par algorithme
VS Instances de la compétition
Instance de la compétition
data_0015 1 0.759736 0.759736 0.7599 1 1 1
data_0014 0.994739 0.989755 0.989807 0.994705 0.994739 0.994393 0.994462
data_0013 0.999992 0.999945 1 0.999998 0.999992 0.999996 0.999999
data_0012 0.993813 0.990531 0.990555 0.993207 0.993813 0.993758 0.993777
data_0011 0.989761 0.986208 0.986215 0.989408 0.989941 0.989318 0.989146
data_0010 0.997938 0.996037 0.996056 0.997905 0.997885 0.997821 0.997737
data_0009 0.928529 0.512232 0.512232 0.518426 0.928529 0.928767 0.928767
data_0008 0.995339 0.991106 0.990528 0.995861 0.995096 0.995469 0.993717
data_0007 1 0.999989 1 1 1 1 1
data_0006 0.654822 0.654822 0.654822 0.654822 0.654822 0.654822 0.654822
data_0005 0.997948 0.989234 0.989234 0.997071 0.9979 0.997216 0.996445
data_0004 0.998701 0.971519 0.971122 0.994495 0.998701 0.995942 0.983095
data_0003 1 1 1 1 1 1 1
data_0002 0.996346 0.982312 0.981809 0.994966 0.998504 0.993548 0.9906
data_0001 0.996658 0.643205 0.643191 0.783159 0.97958 0.904562 0.912247
contestScore
Greedy90
GreedyBase
GreedyRR90_10_100
GreedyRRK85_5_85
GreedyRRK90_10_100
GreedyRRK90_3_100
Algorithme
Best Score Ratio
1.0
0.9
0.8
0.7
0.6

Métriques : Groupement par ACP



$Carrés \equiv Non\ Compétitifs$
$Triangles \equiv Compétitifs$



« Grandes » Instances

- Grandes tailles
- Structure proportionnée

Instances « générales »

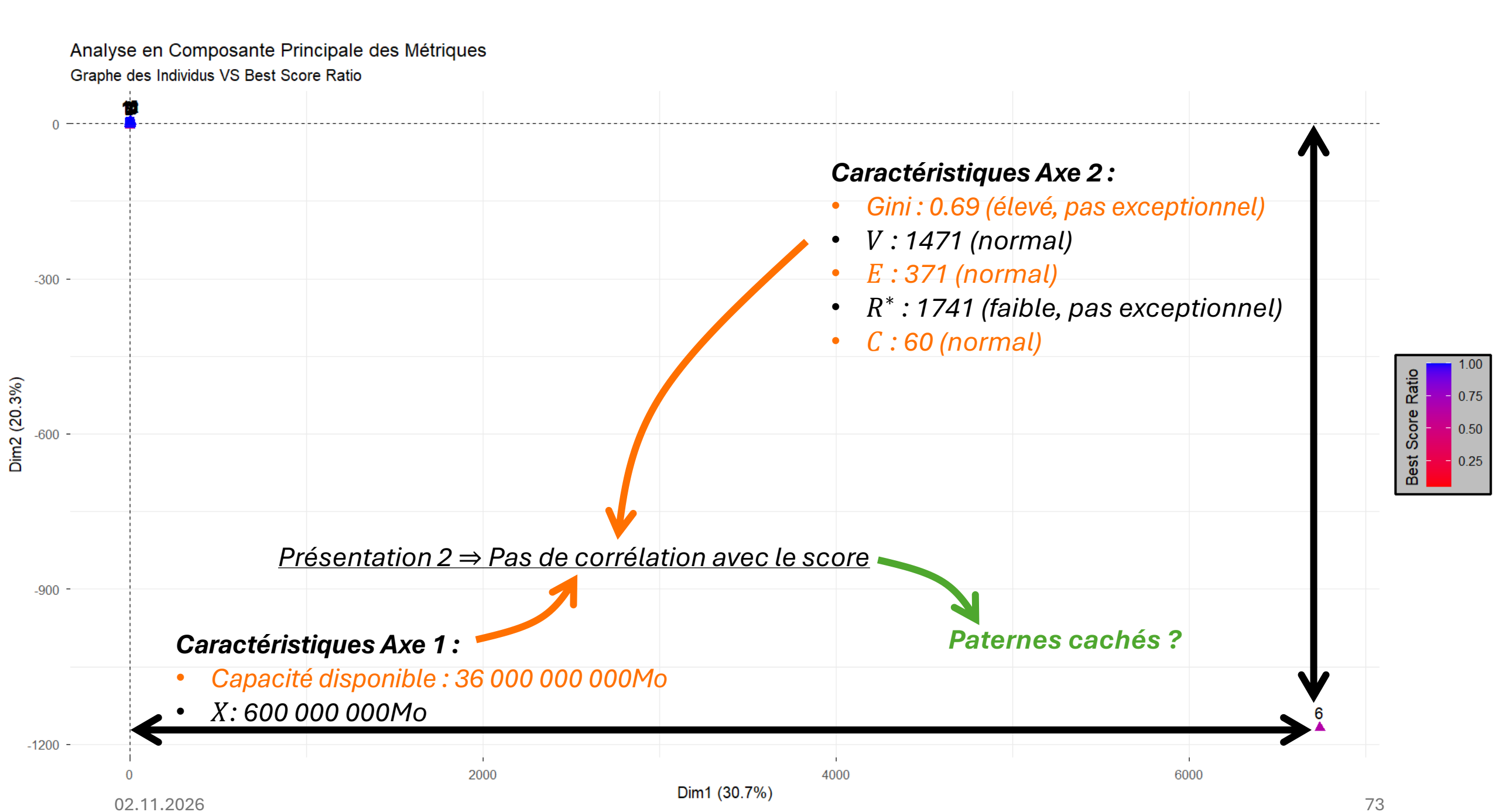
- Sans caractéristiques particulières

Instances difficiles

- Petites instances
- Force un choix (petite VS grande vidéo sur cache à capacité limité)

Instances Permissives

- $X > 200k$
- $\mathbb{E}[\#V]\ par\ cache > 390$
- $Pression > 3.9$


Analyse en Composante Principale des Métriques
Graphe des Individus VS Best Score Ratio
Caractéristiques Axe 2 :
Gini : 0.69 (élevé, pas exceptionnel)
V : 1471 (normal)
E : 371 (normal)
R* : 1741 (faible, pas exceptionnel)
C : 60 (normal)
Présentation 2 ⇒ Pas de corrélation avec le score
Paternes cachés ?
Caractéristiques Axe 1 :
Capacité disponible : 36 000 000 000Mo
X: 600 000 000Mo
Best Score Ratio
1.00
0.75
0.50
0.25
Dim2 (20.3%)
Dim1 (30.7%)
0
-300
-600
-900
-1200
0
2000
4000
6000
6

[redacted] [redacted] 29/10/2026 16:57

hello arno

oui c'est sûrement la notre

l'idée c'était que l'optimal se trouve en classant par nb de requete (notre approche)

et au contraire ne tende pas vers l'optimal en classant par gain potentiel (l'approche des autres groupes)

c'est [redacted] qui s'en ai occupé donc je suis pas sûre mais on parlait aussi de mettre la taille des caches et le nb de requête à l'interieur des requetes de la forme pgcd enorme * petite valeur

comme ça pour ceux qui veulent faire un sac à dos en dynamique il sont des enomes valeurs et poids donc c'est super long

mais nous on divise par le pgcd donc c'est plus petit et plus rapide

SWOT & Conclusion

Analyse SWOT

Greedy par densité → Ruin & Recreate ↔ Knapsack

Strengths

- Greedy produit un bon résultat initial
- Knapsack exact : jusqu'à +81 % (instance09)
- Interruptible à tout moment, sans perte

Weaknesses

- Knapsack exact par cache, pas entre caches
- Gain marginal quand le greedy est déjà quasi optimal
- Fort coût niveau computation
- Nos algorithmes peuvent êtres sujets aux optimum locaux

Opportunities

- Paramètres adaptés au cluster d'instance
- Knapsack ciblé sur les caches cassées

Threats

- Instances pièges faciles à construire, à toute échelle
- Overfitting sur les instances officielles
- Présuppositions concernant la densité
- Résultats partiellement dépendants du hasard